Static analysis and software verification

Lecture 9 - Abstract interpretation in LiSA

Vincenzo Arceri - University of Parma - vincenzo.arceri@unipr.it



UNIVERSITÀ
DI PARMA

LiSA core (2020 - now)



Luca
Negrini
(UNIVE)



Vincenzo
Arceri
(UNIPR)

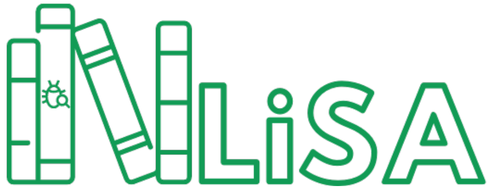


Luca
Olivieri
(UNIVE)



Pietro
Ferrara
(UNIVE)

LiSA, a Library for Static Analysis



Library for creating static analyzers based on abstract interpretation

- For multiple programming languages
- For a variety of different domains

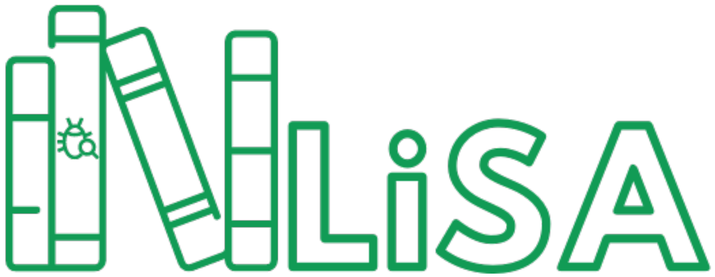
Purposes:

- Experiment with modular implementations
- Be easy to pick-up (simple, close to formalization)
- Fast prototyping of analyzers and domains

Open source Java library

Created and maintained by UNIPR and UNIVE

LiSA is...



LiSA is...



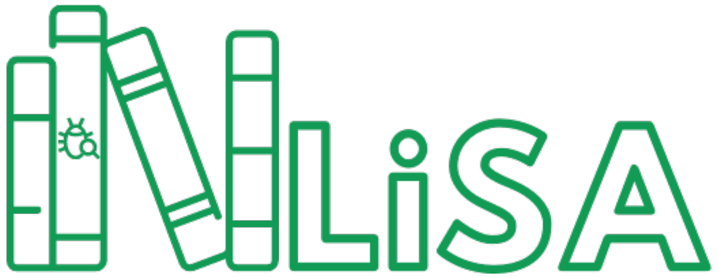
generic analysis engine

pre-developed components

LiSA is...

configuration parameters
components selection
plug-in new components



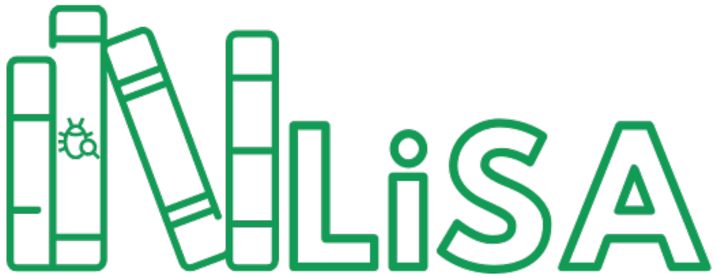


generic analysis engine
pre-developed components

LiSA is...

configuration parameters
components selection
plug-in new components



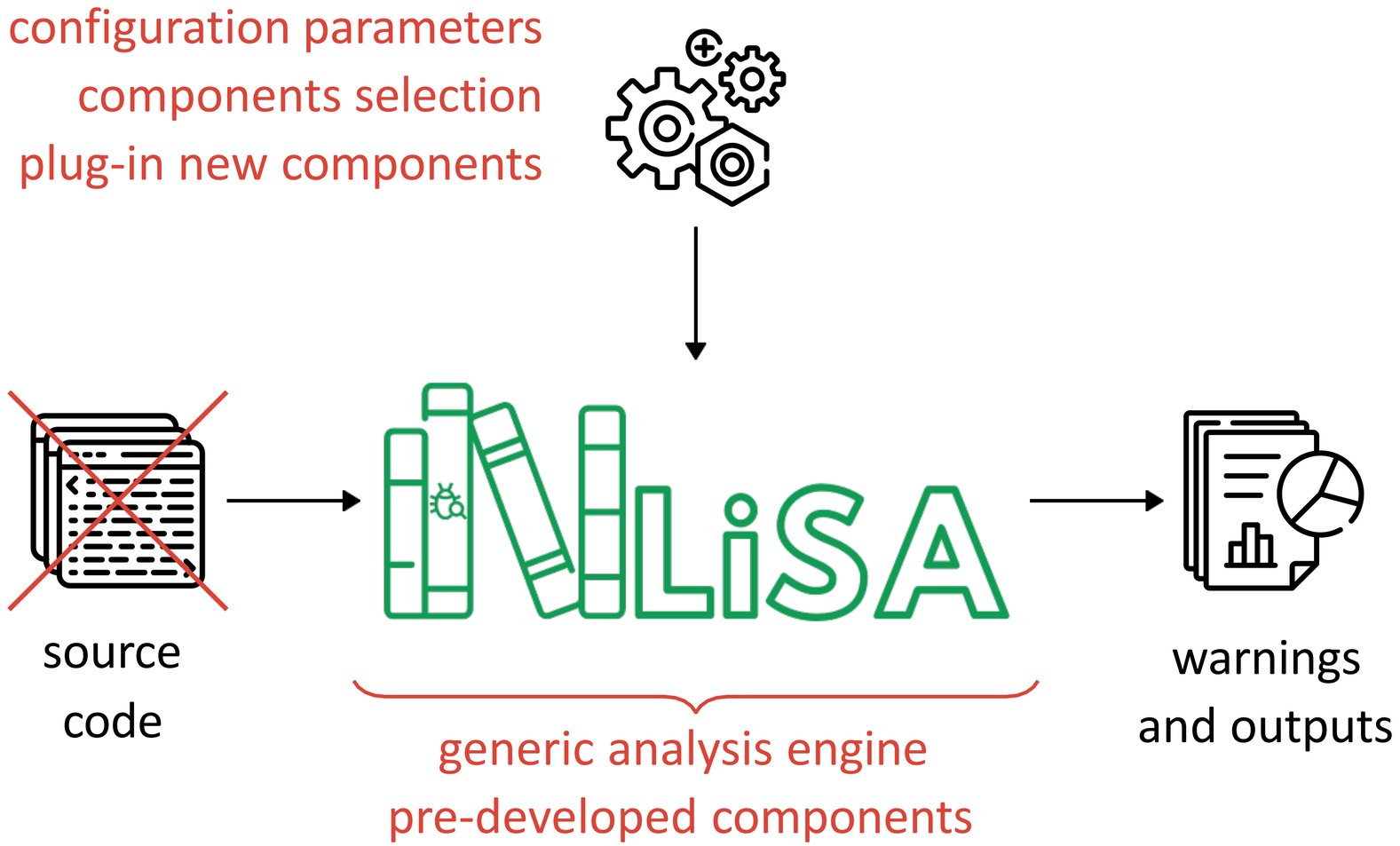




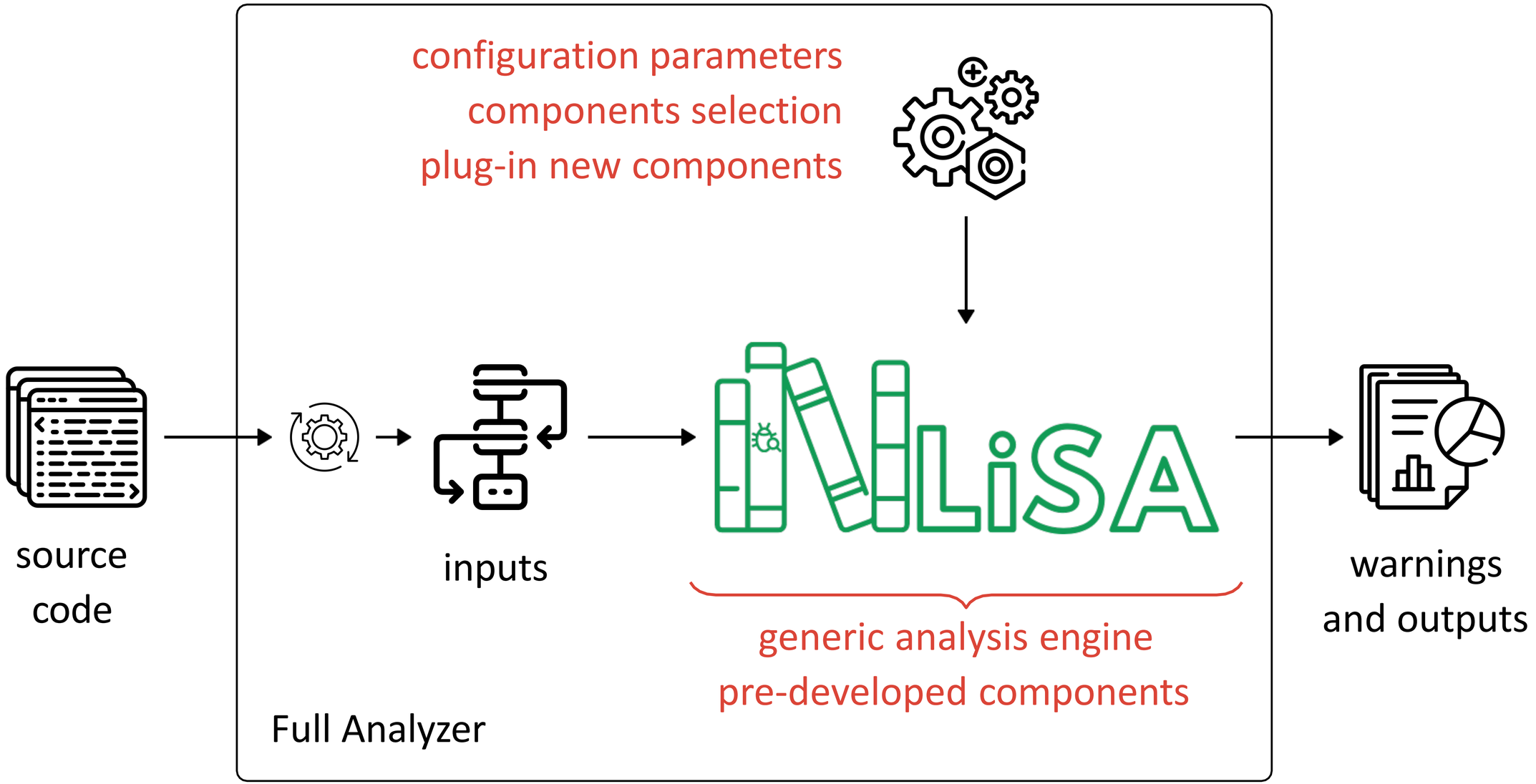
warnings
and outputs

generic analysis engine
pre-developed components

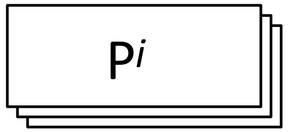
LiSA is... Not a Full Analyzer



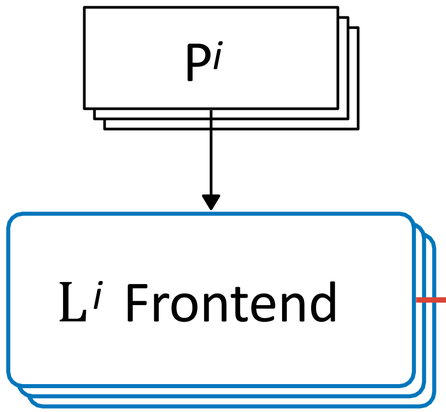
LiSA is... a Library



LiSA Overview



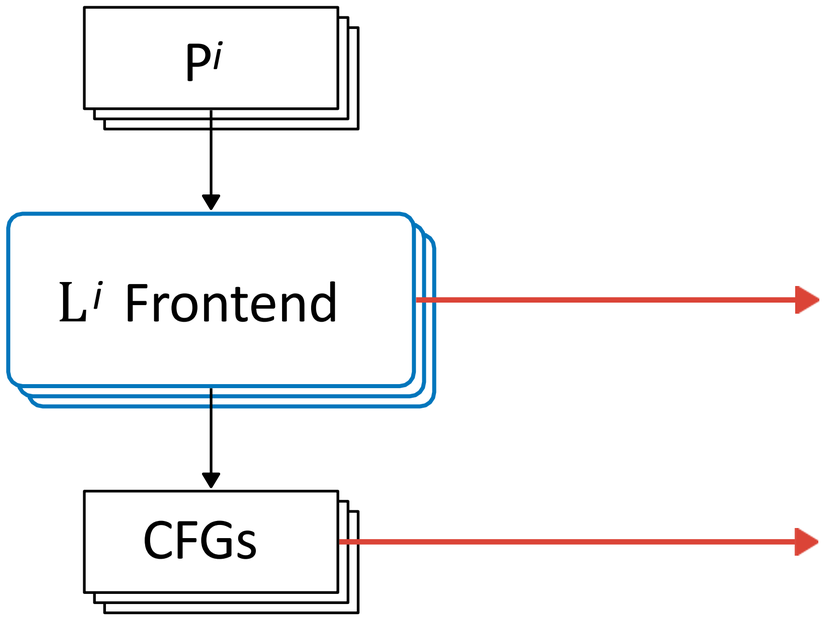
LiSA Overview



Compilation to LiSA program
Definition of the language **execution model**
Definition of **instruction semantics**

✷ language dependent

LiSA Overview



Compilation to LiSA program

Definition of the language **execution model**

Definition of **instruction semantics**

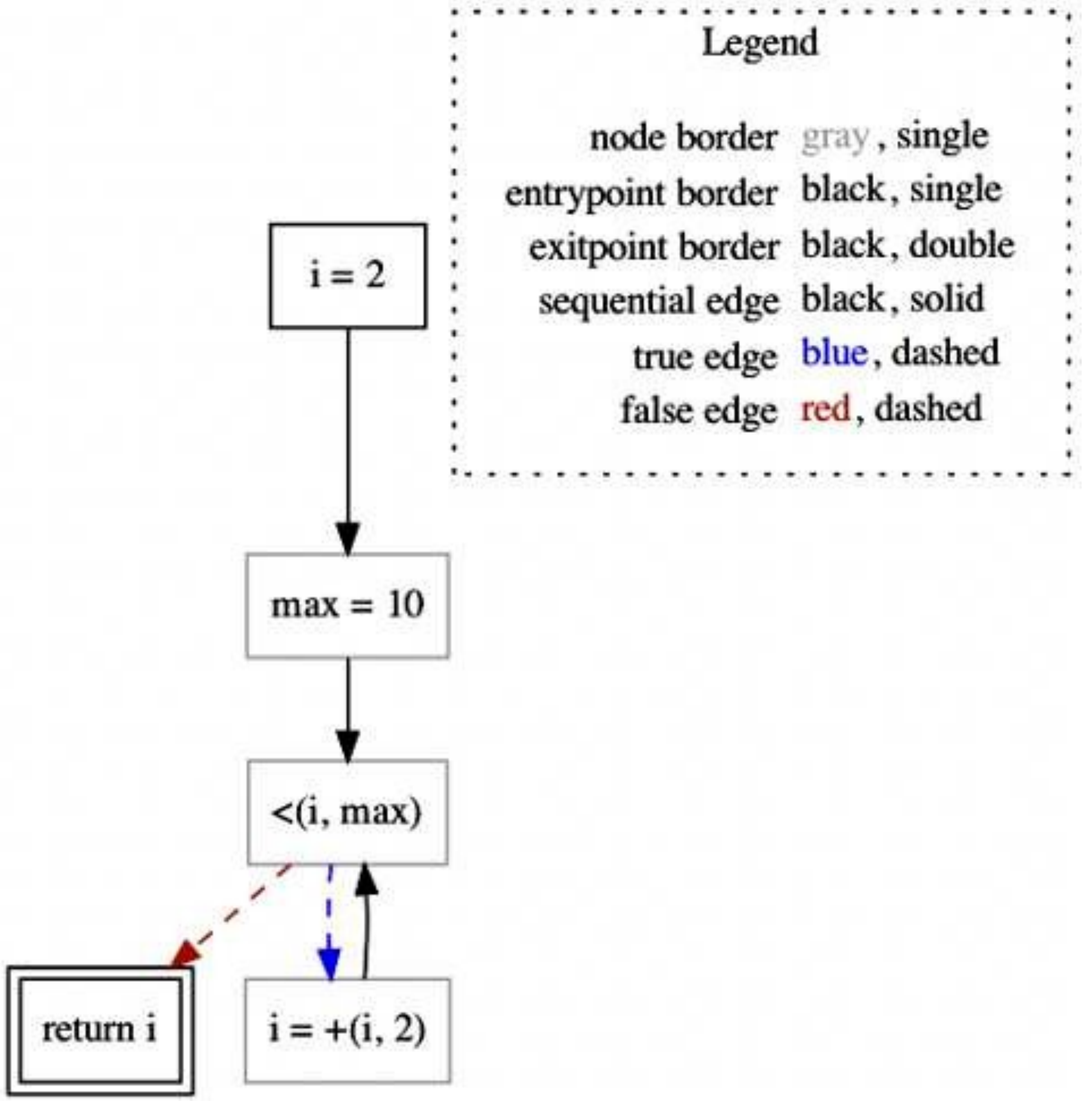
Embedded control flow

Standard fixpoints

✷ language dependent

CFGs

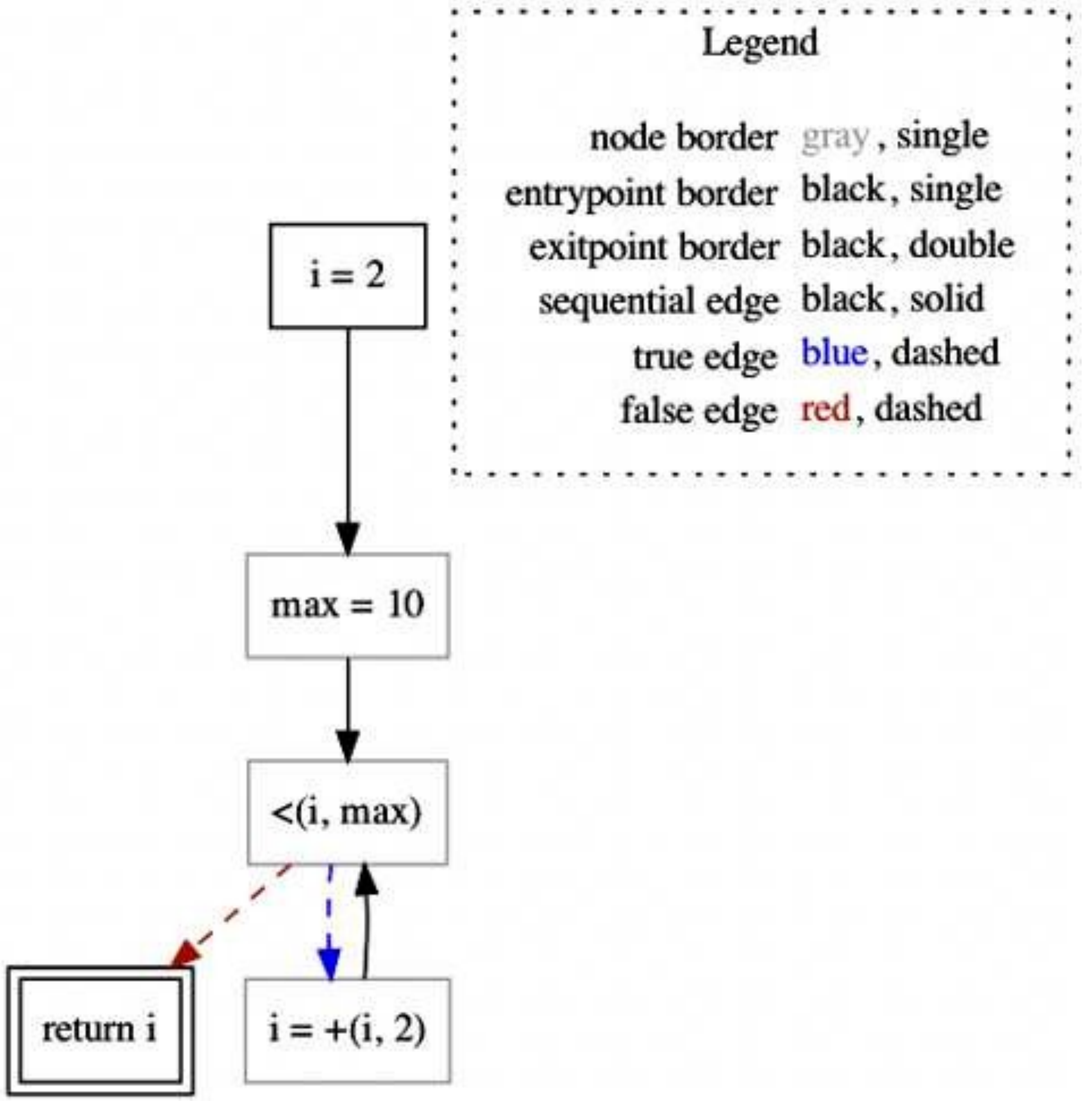
```
1 int i = 2;
2 int max = 10;
3 while (i < max)
4   i = i + 1;
5 return i;
```



CFGs

```
1 int i = 2;
2 int max = 10;
3 while (i < max)
4   i = i + 1;
5 return i;
```

Nodes are **Statement** instances

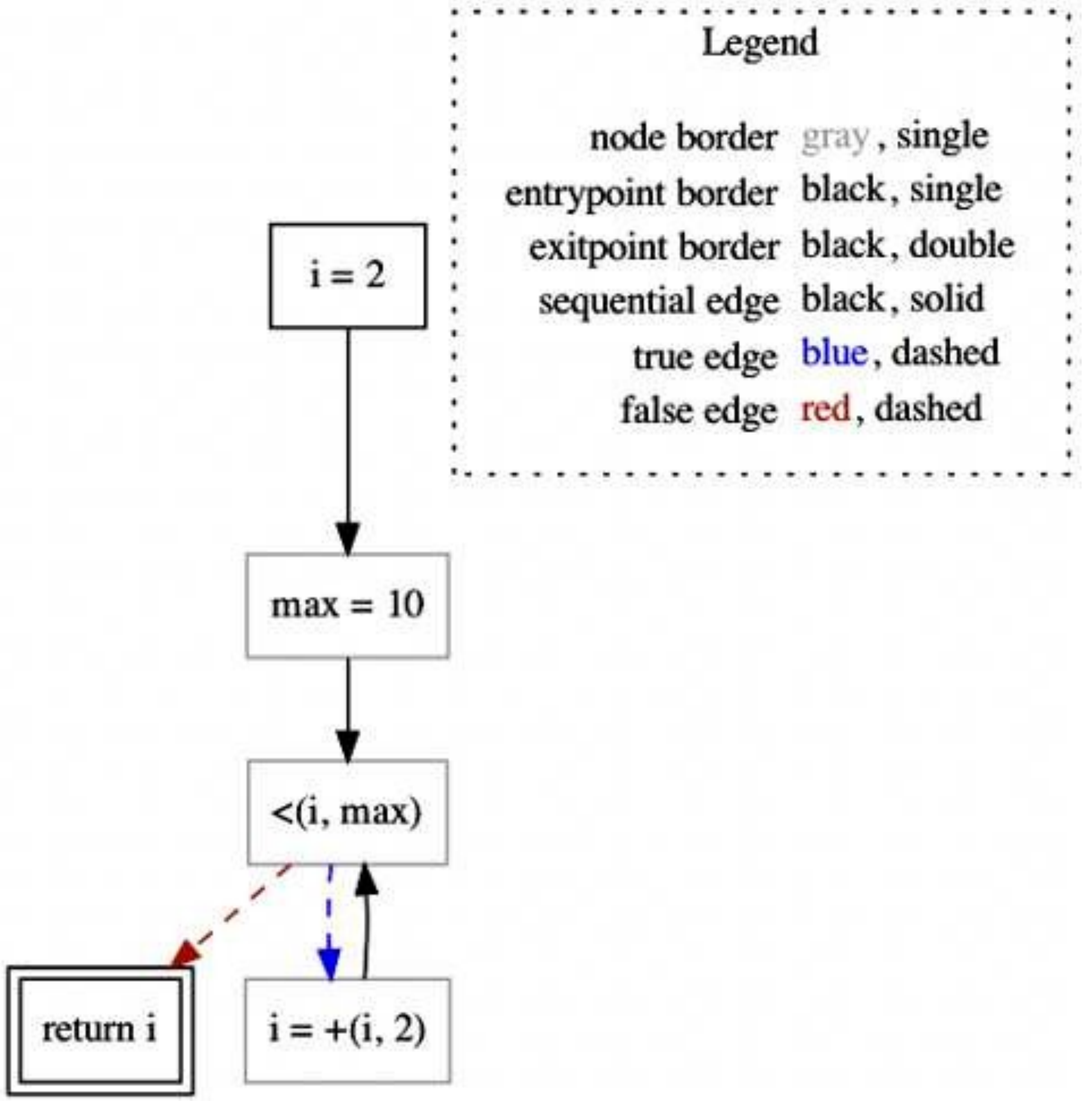


CFGs

```
1 int i = 2;
2 int max = 10;
3 while (i < max)
4   i = i + 1;
5 return i;
```

Nodes are **Statement** instances

Edges are **Edge** instances



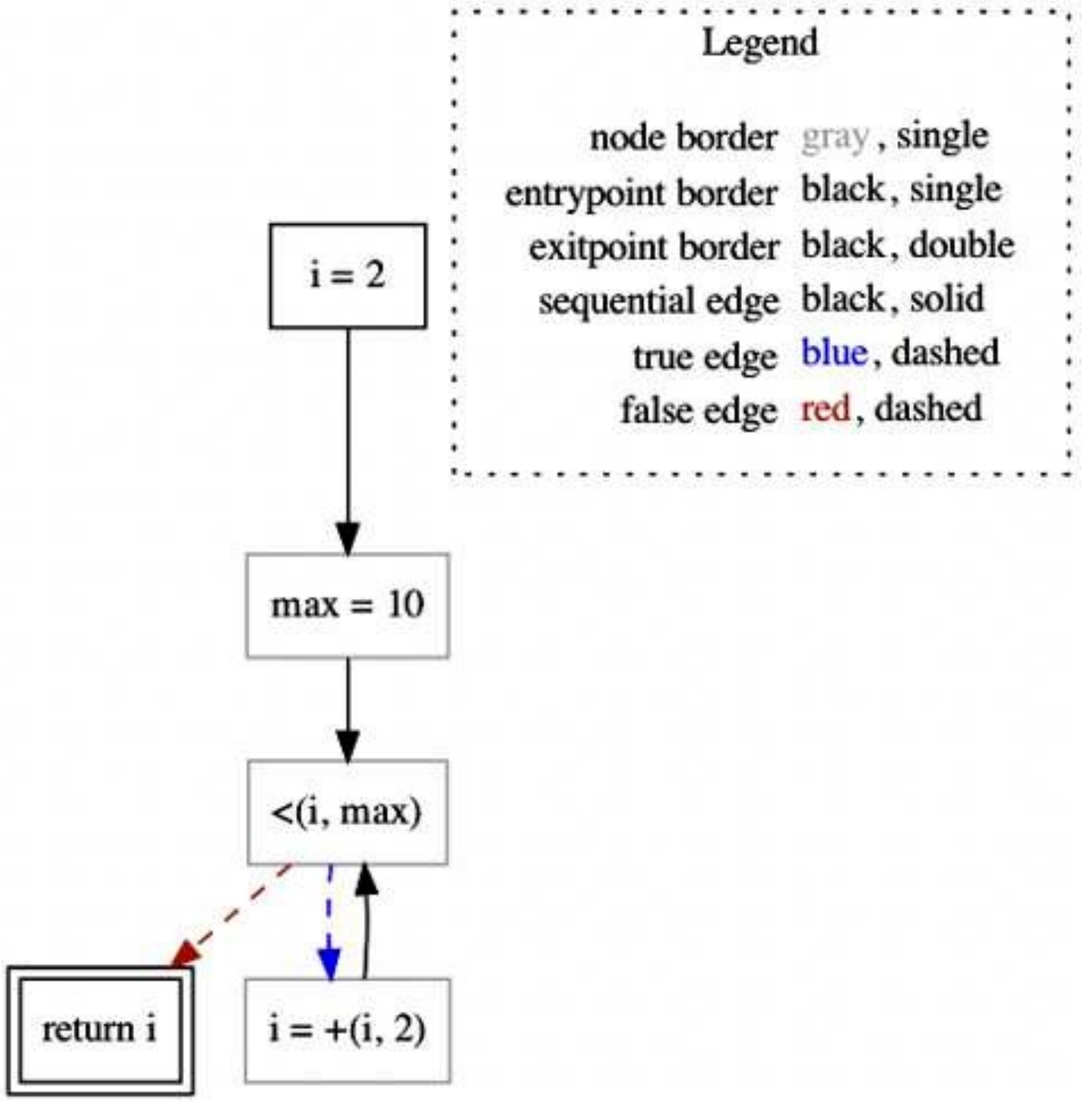
CFGs

```
1 int i = 2;
2 int max = 10;
3 while (i < max)
4   i = i + 1;
5 return i;
```

Nodes are **Statement** instances

Edges are **Edge** instances

LiSA does not fix a semantics for any **Statement**/**Edge**, but lets users define them (more details later)



Computing Fixpoints on CFGs

1 **forall** $n \in N$ **do**
2 $\quad out_n \leftarrow \bot$;
3 $in_{n_0} \leftarrow \top$;
4 $out_{n_0} \leftarrow \texttt{semantics}_{n_0}(in_{n_0})$;
5 $ws \leftarrow \texttt{succ}(n_0)$;
6 **while** $ws \neq \emptyset$ **do**
7 $\quad n \leftarrow \texttt{pop}(ws)$;
8 $\quad in_n \leftarrow \bigsqcup\{\texttt{traverse}_{m \to n}(out_m) :$
9 $\qquad\qquad m \in \texttt{preds}(n)\}$;
10 $\quad tmp_n \leftarrow \texttt{semantics}_n(in_n)$;
11 $\quad$ **if** $tmp_n \not\sqsubseteq out_n$ **then**
12 $\quad\quad out_n \leftarrow out_n \oplus tmp_n$;
13 $\quad\quad ws \leftarrow ws \cup \texttt{succ}(n)$;
14 **return** *out*;

$\bot$ for not yet processed/unreachable

$\top$ for no information

use `Statement.semantics()`

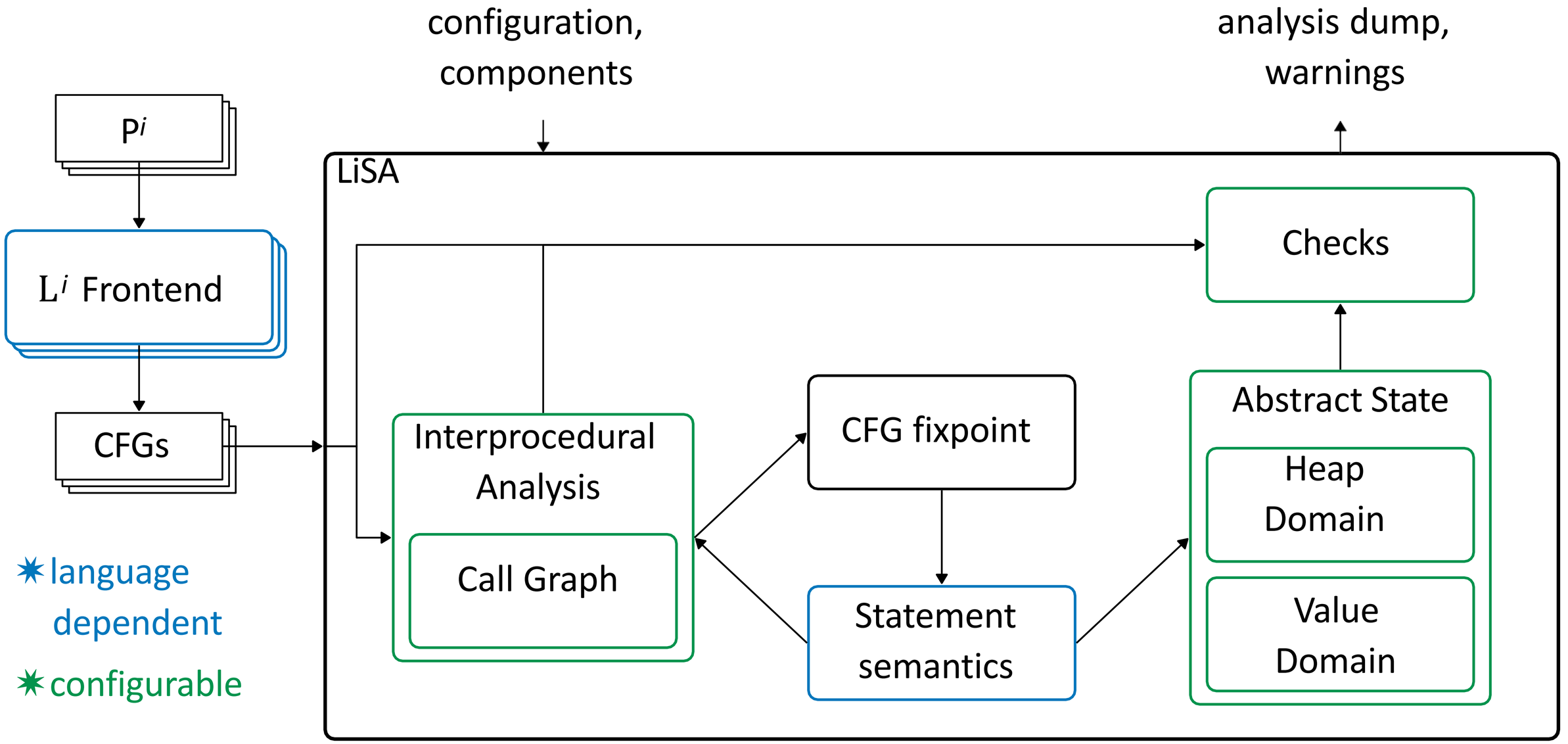
$\bigsqcup$ to over-approximate entry states

use `Edge.traverse()`

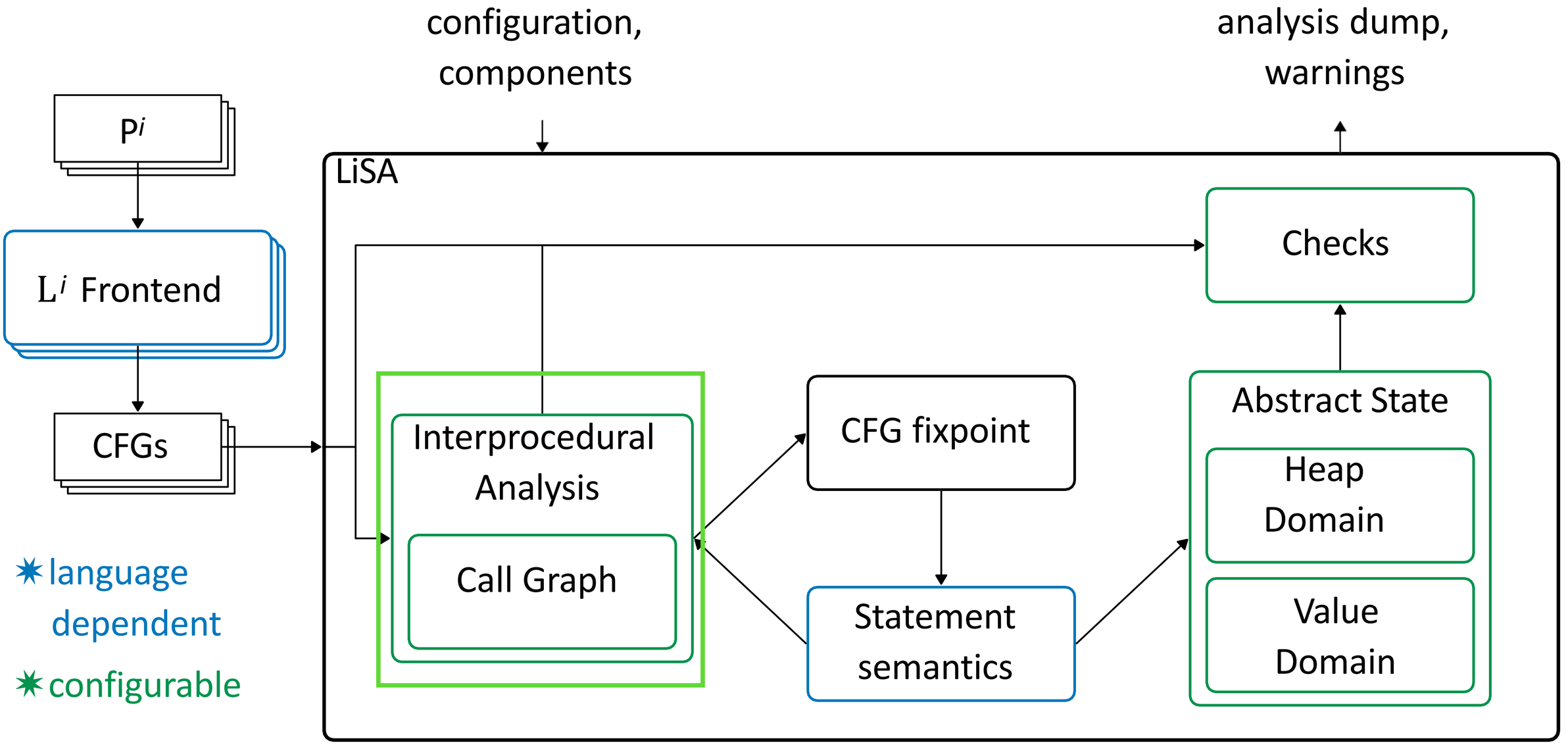
$\sqsubseteq$ to keep “highest” result

$\bigsqcup$ to move “upwards”/∇ for convergence

LiSA Overview



LiSA Overview

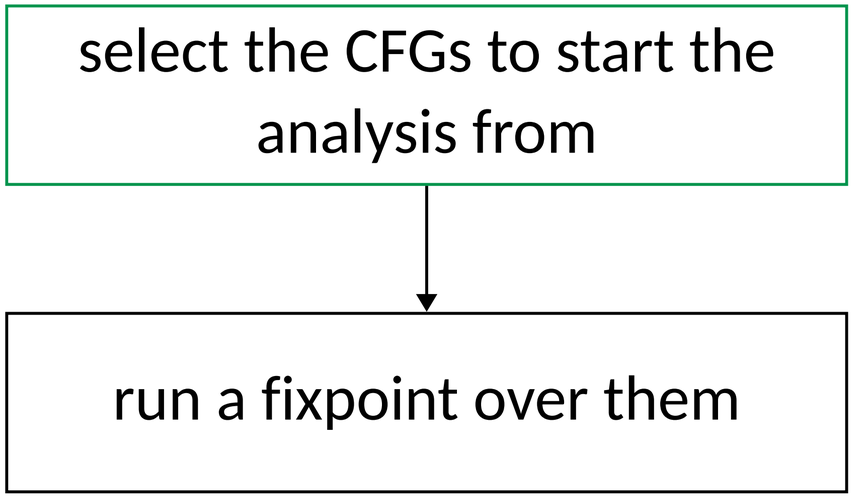


The Program-wide Fixpoint

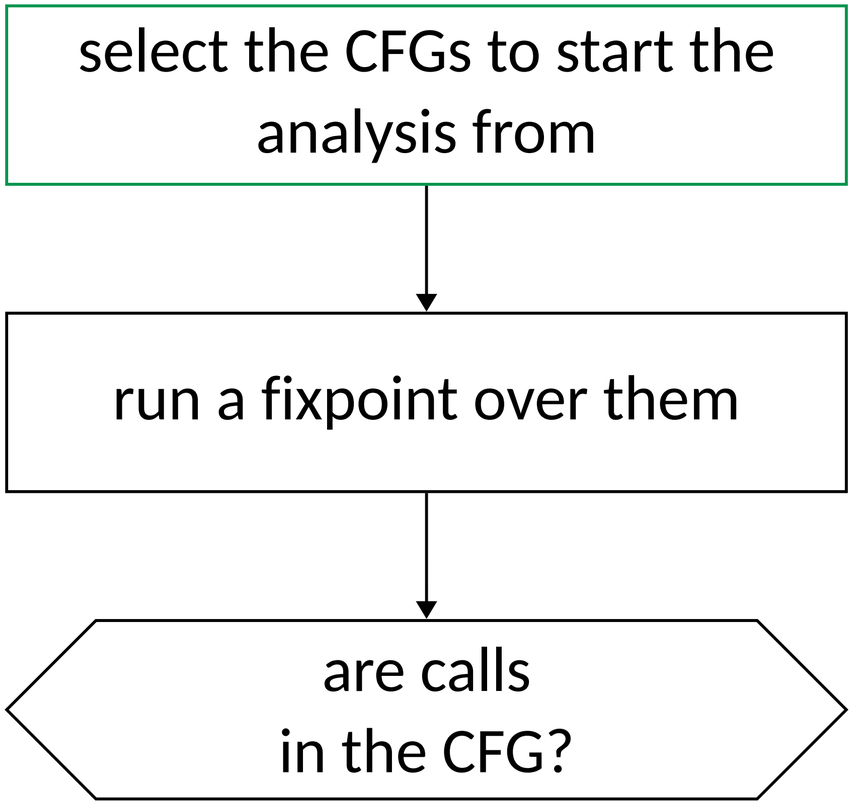
The Program-wide Fixpoint

select the CFGs to start the analysis from

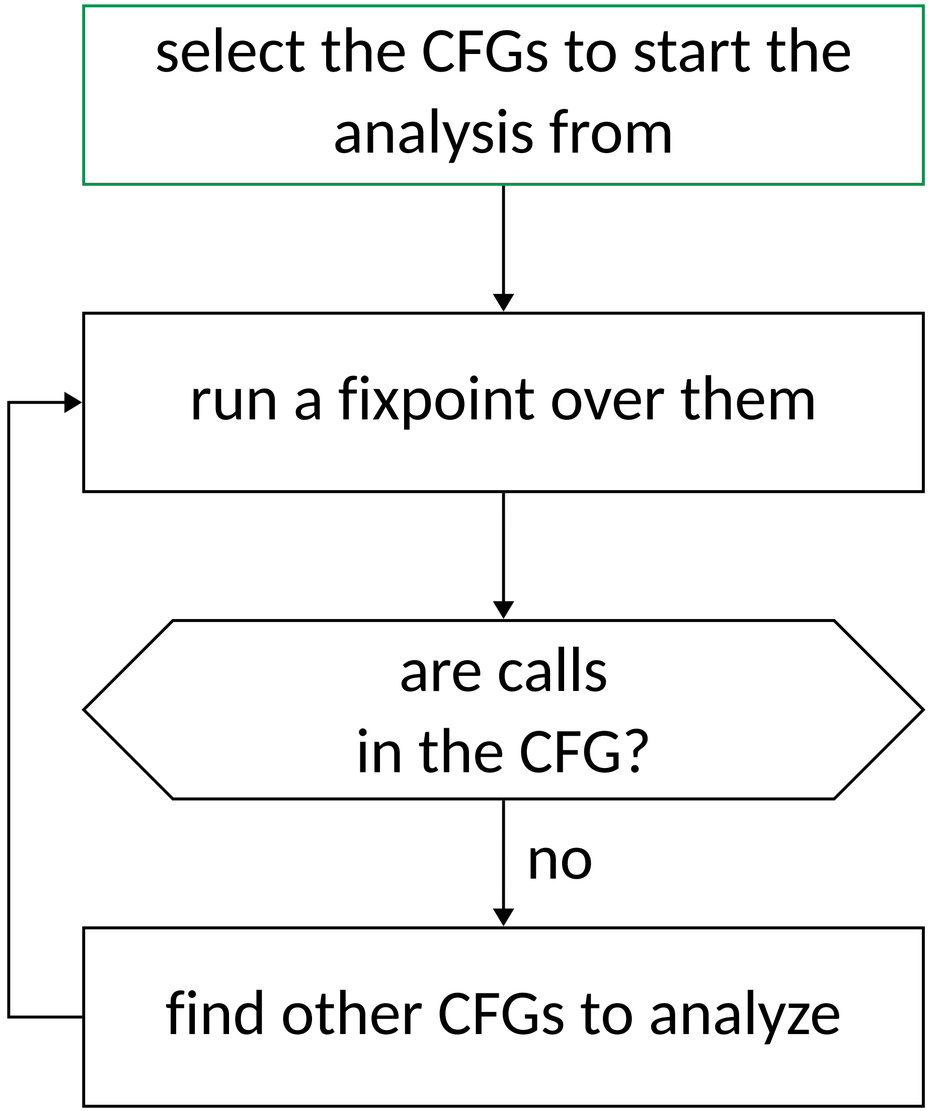
The Program-wide Fixpoint



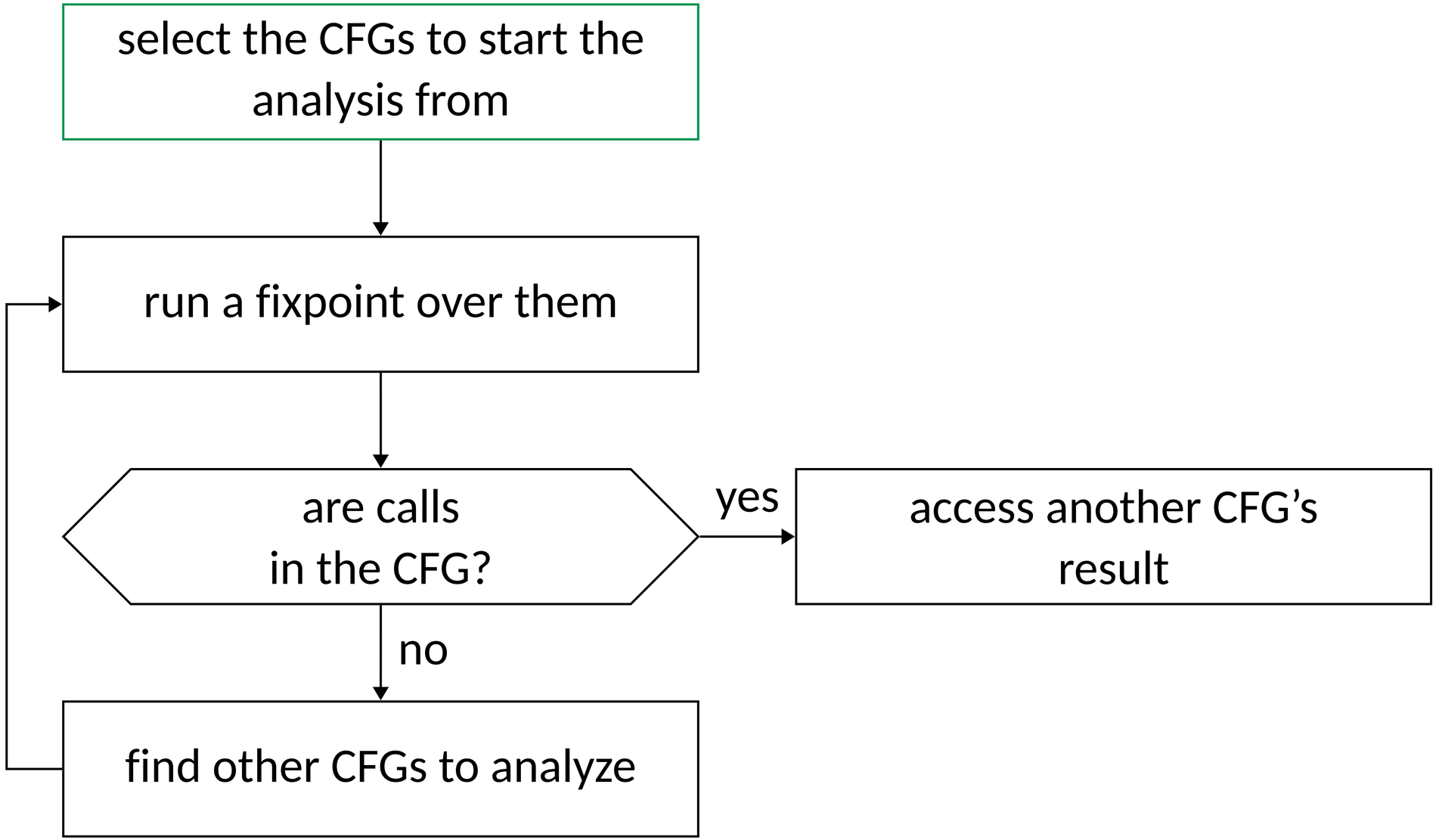
The Program-wide Fixpoint



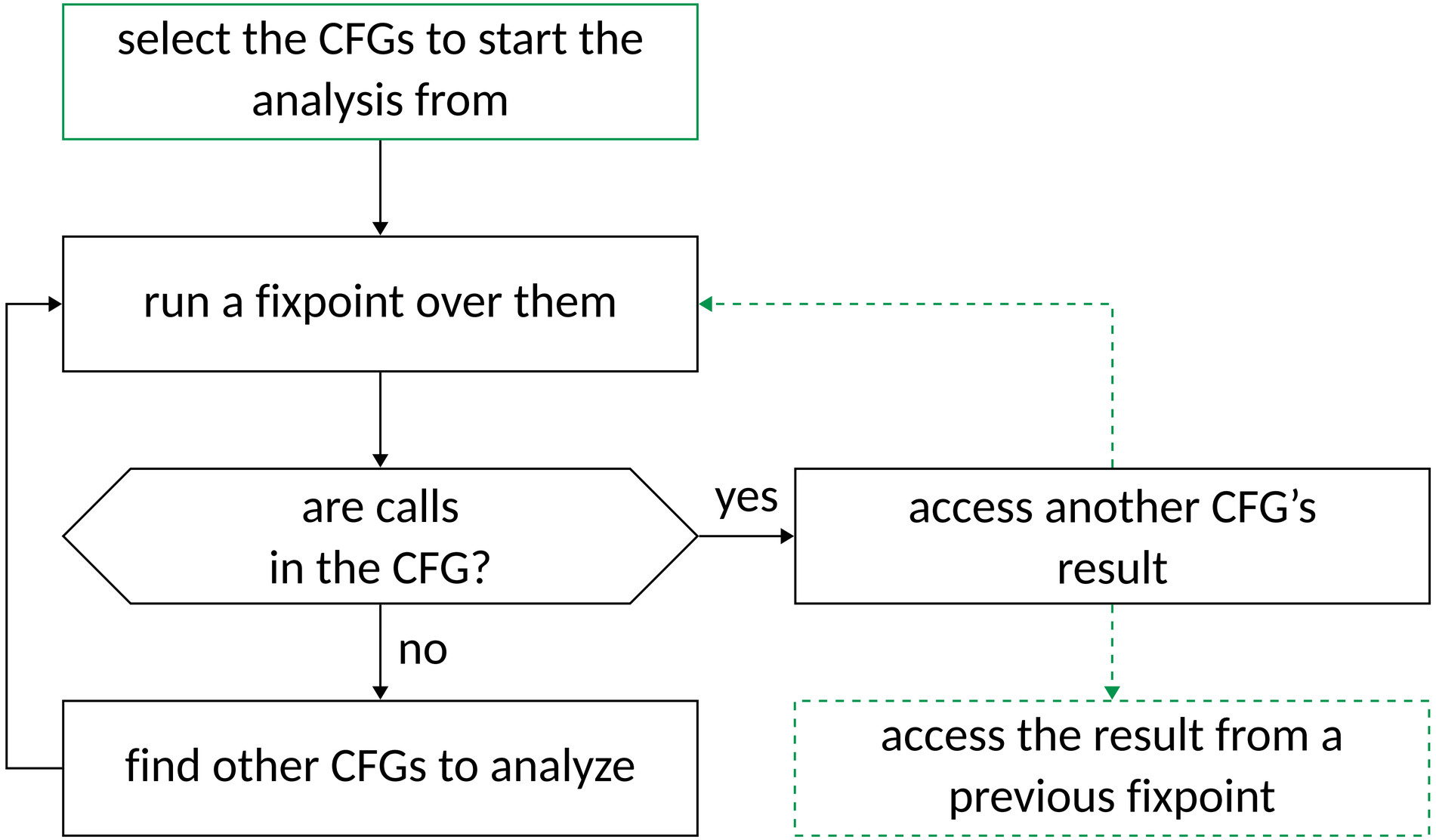
The Program-wide Fixpoint



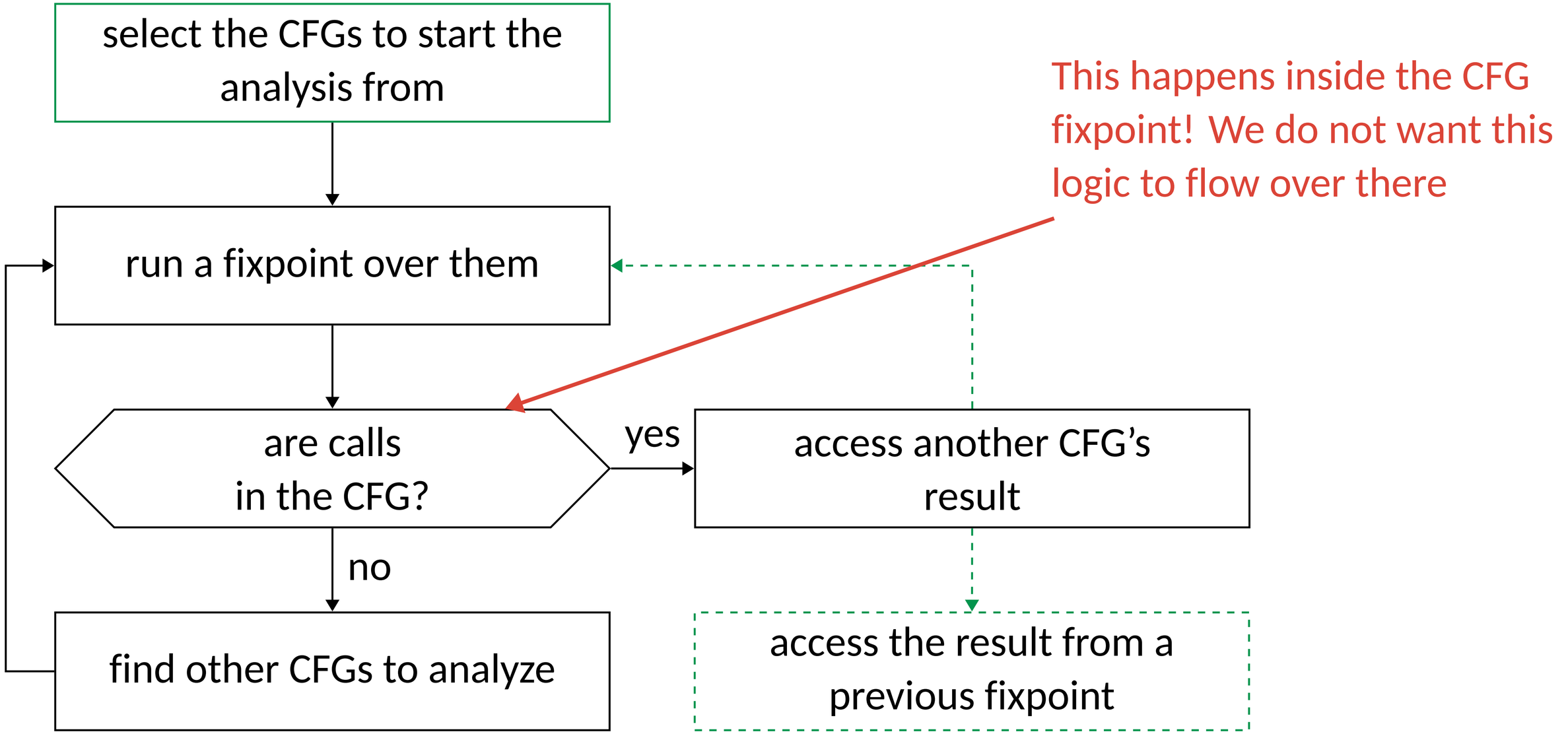
The Program-wide Fixpoint



The Program-wide Fixpoint



The Program-wide Fixpoint



The Interprocedural Analysis

Interprocedural Analysis

☐ configurable

The Interprocedural Analysis



Logic can be:

- Consider CFGs in isolation
- Start at main and follow calls
- …

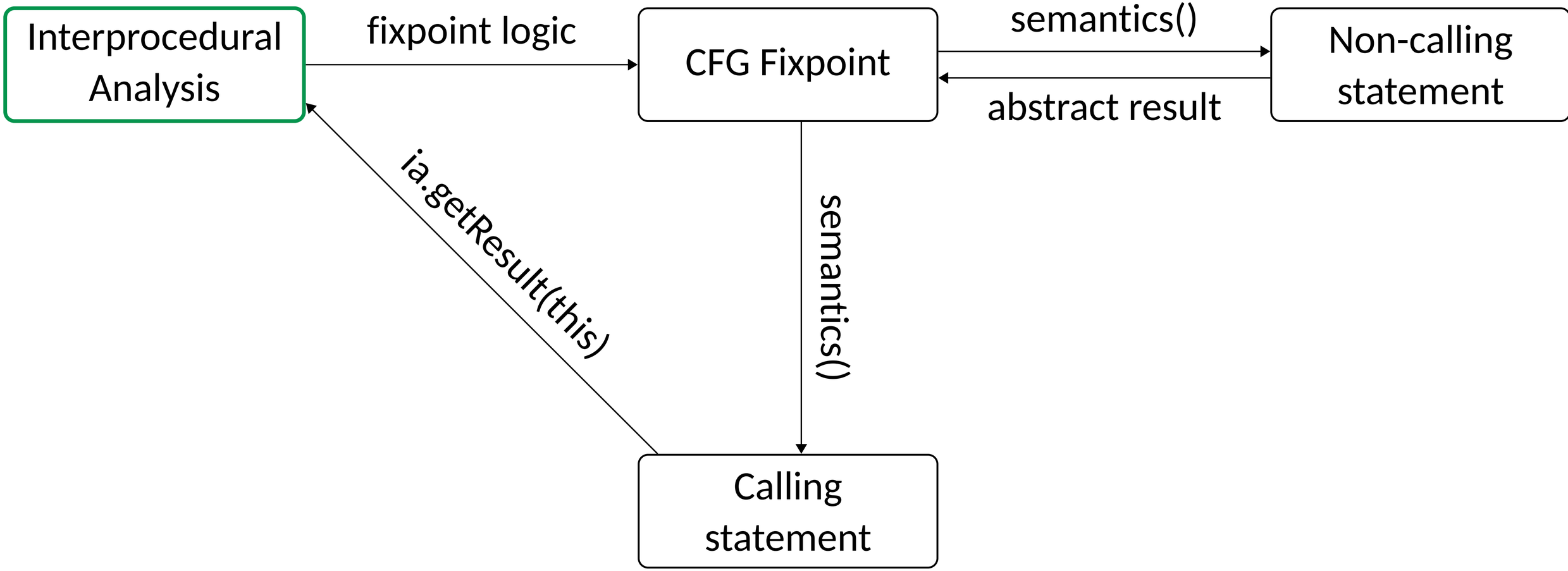
☐ configurable

The Interprocedural Analysis



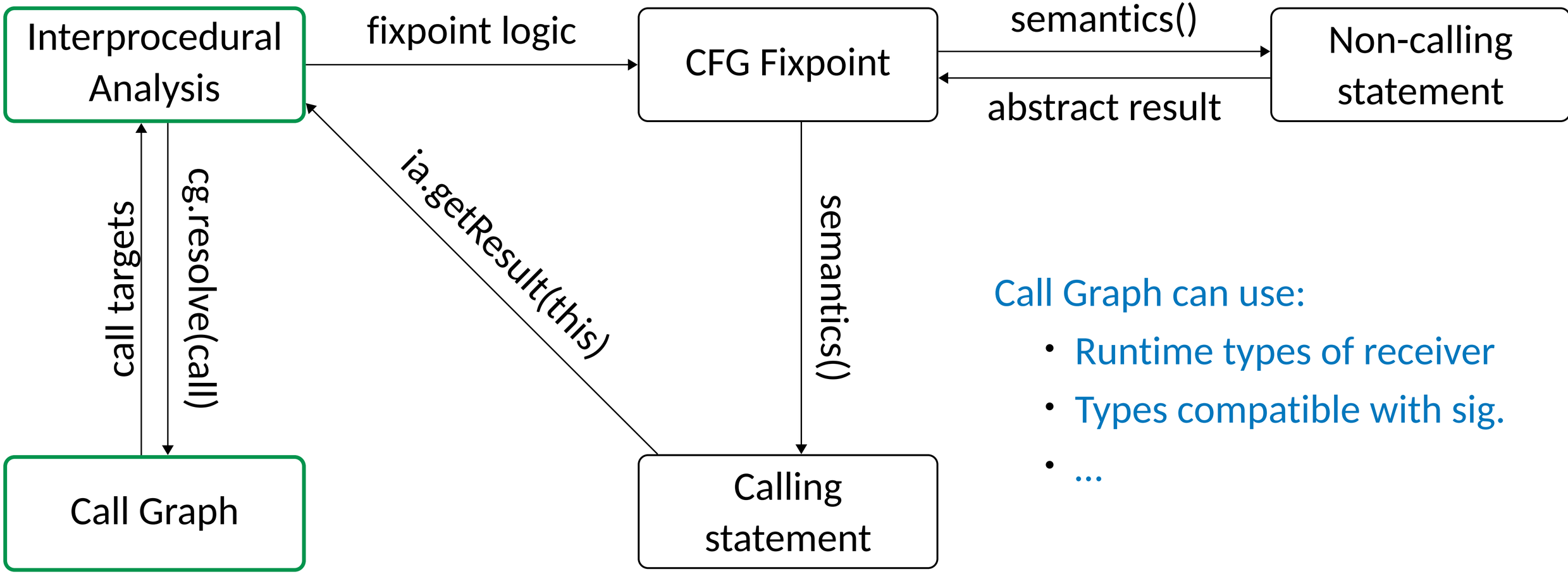
☐ configurable

The Interprocedural Analysis



☐ configurable

The Interprocedural Analysis

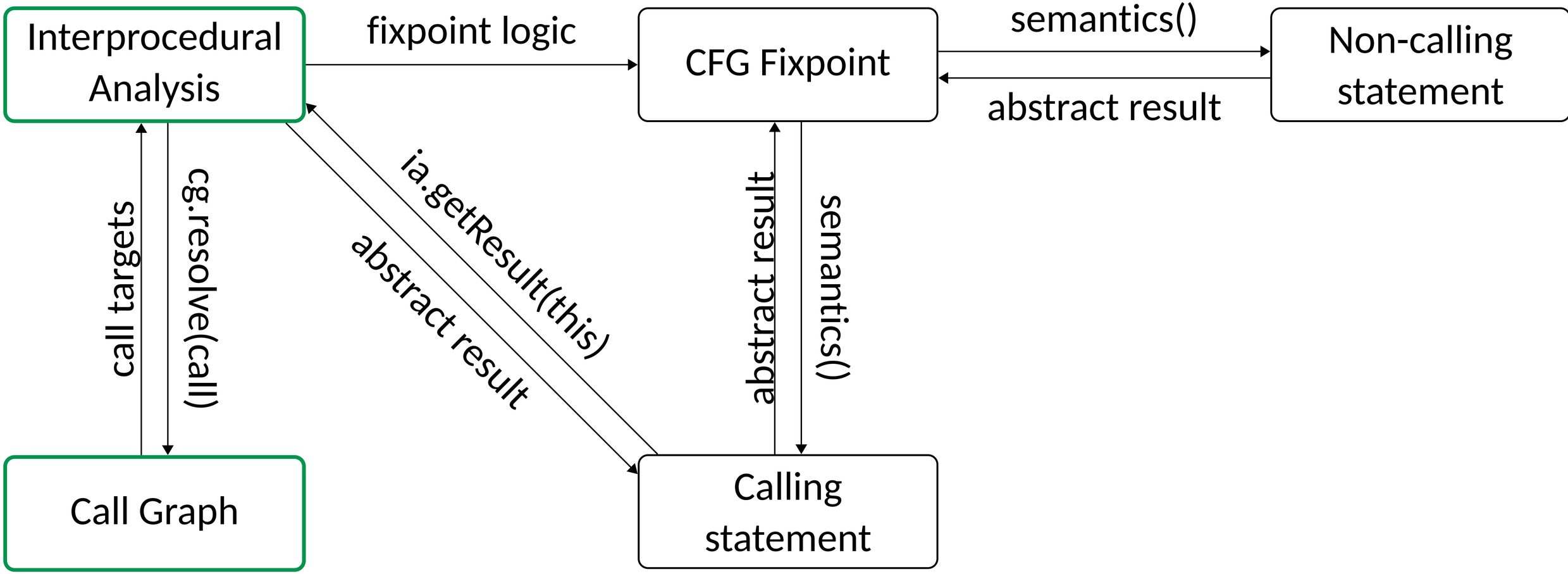


Call Graph can use:

- Runtime types of receiver
- Types compatible with sig.
- …

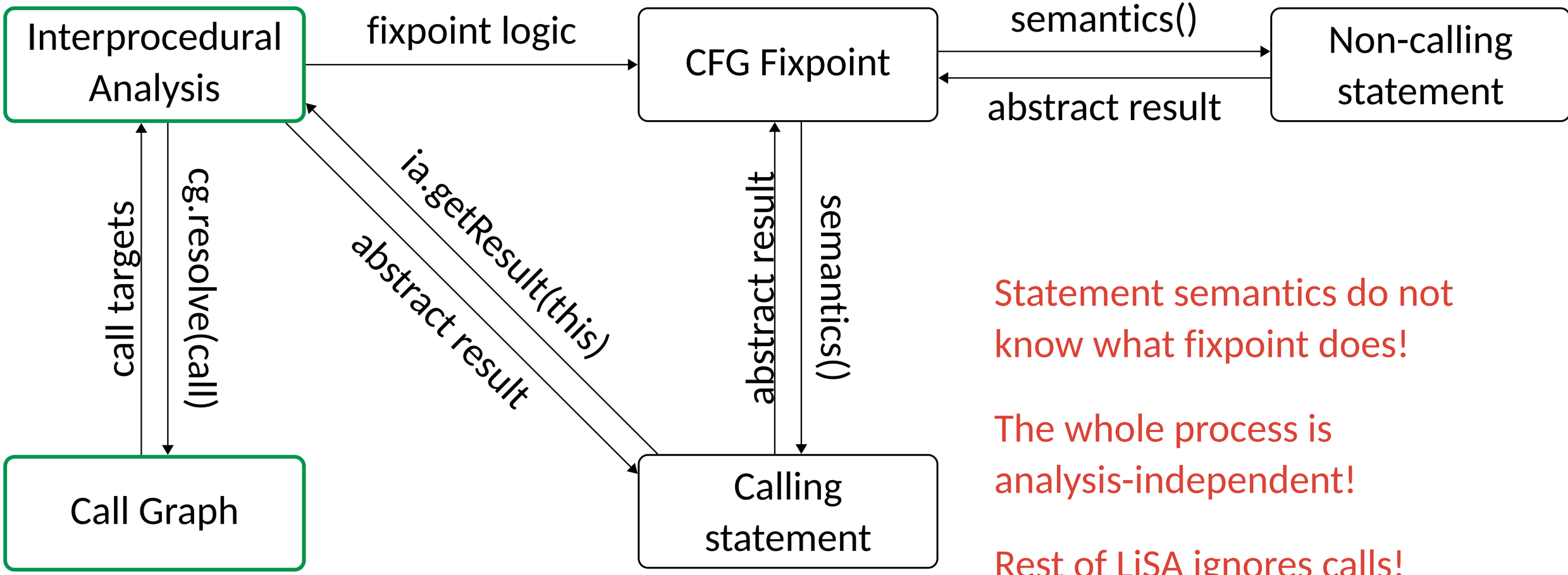
☐ configurable

The Interprocedural Analysis



- configurable

The Interprocedural Analysis



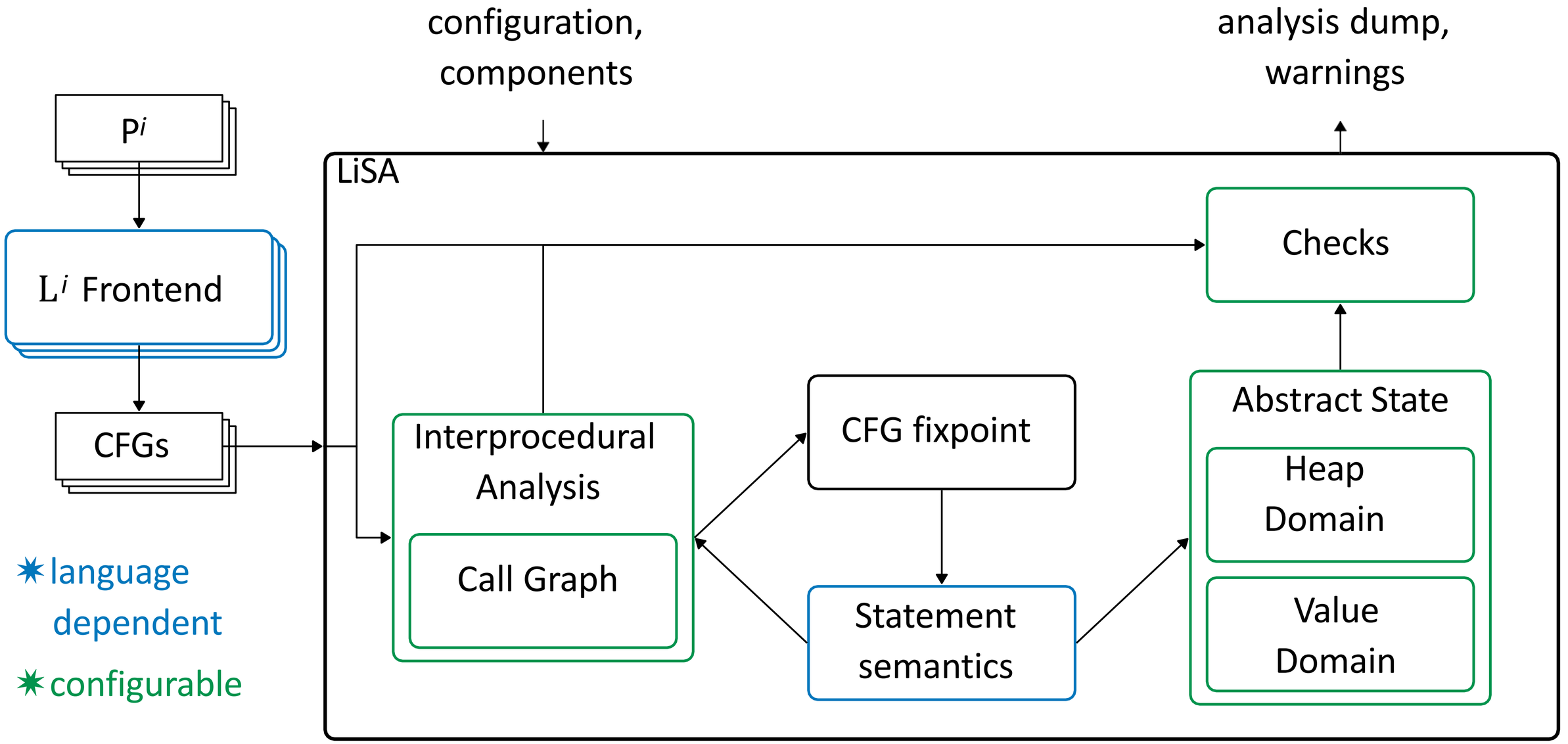
Statement semantics do not know what fixpoint does!

The whole process is analysis-independent!

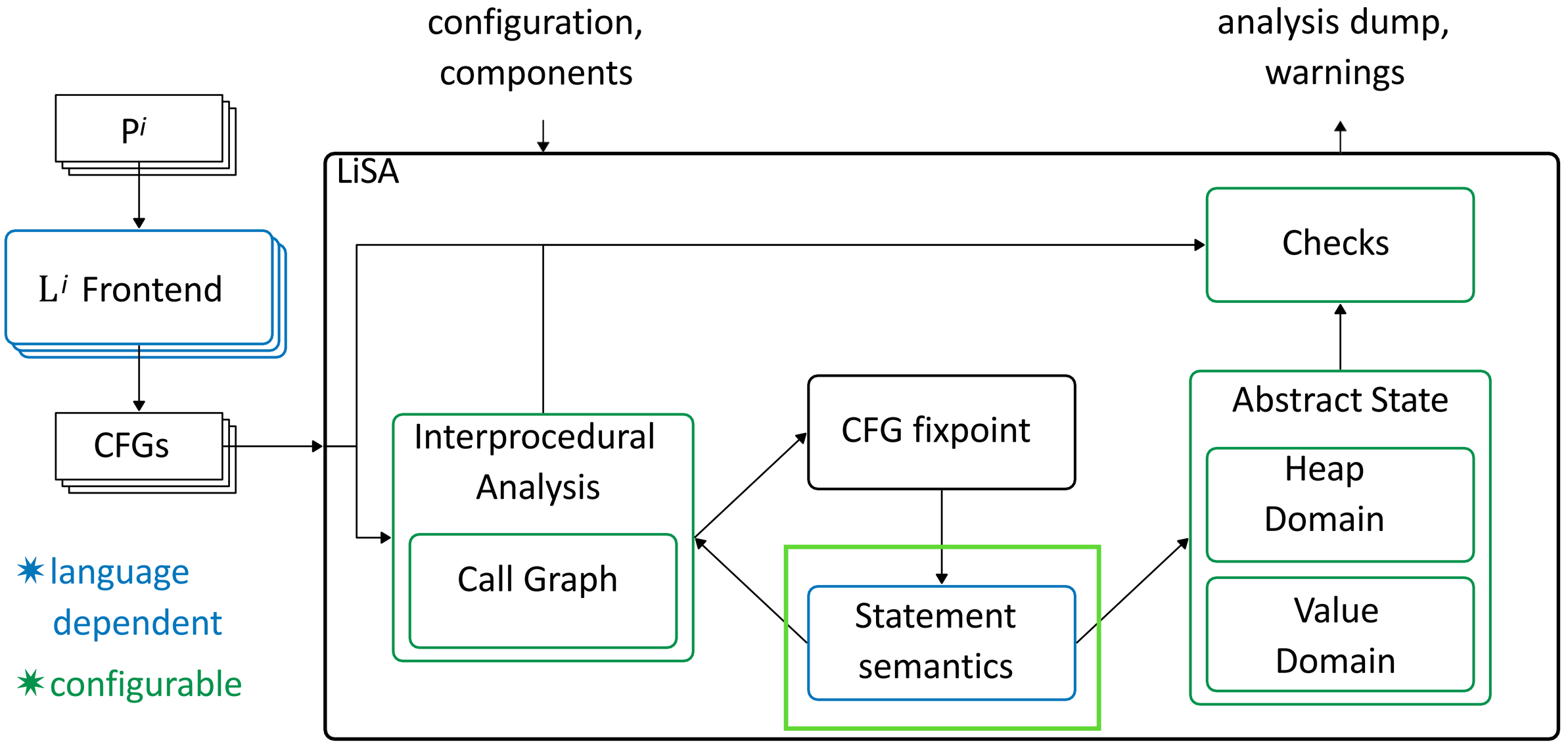
Rest of LiSA ignores calls!

☐ configurable

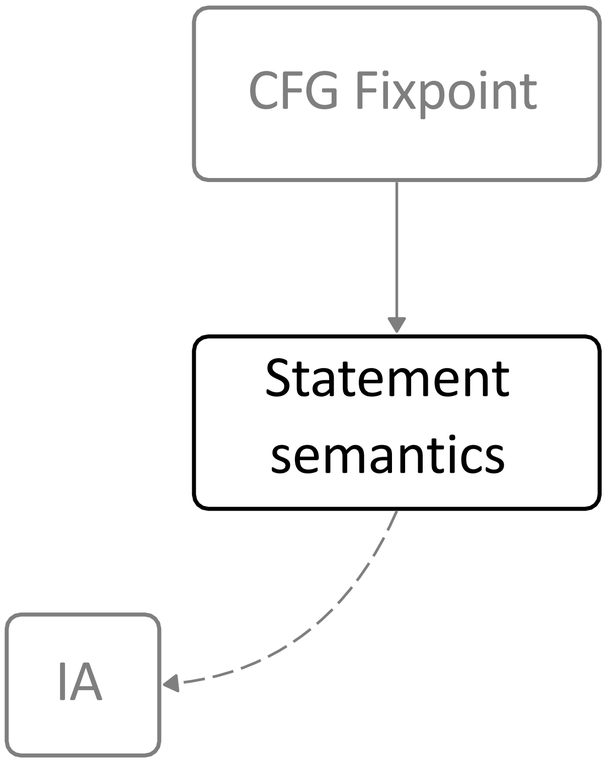
LiSA Overview



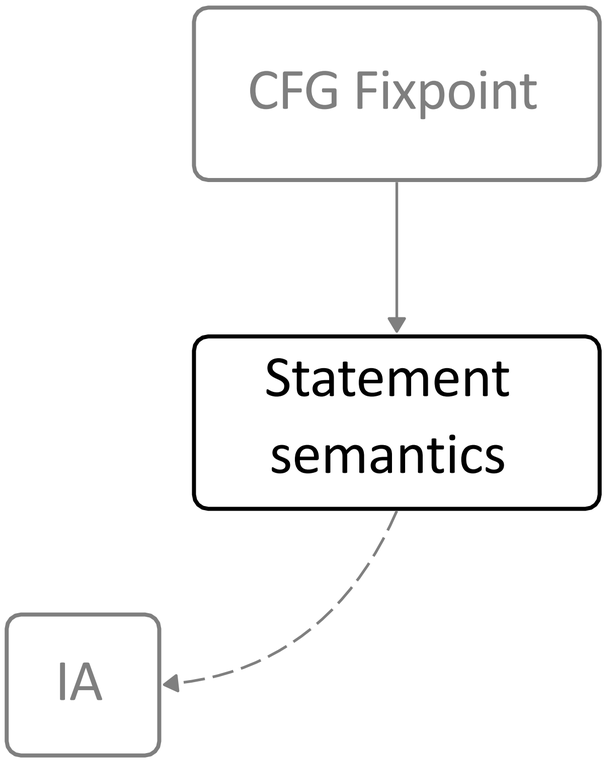
LiSA Overview



Syntax vs Semantics



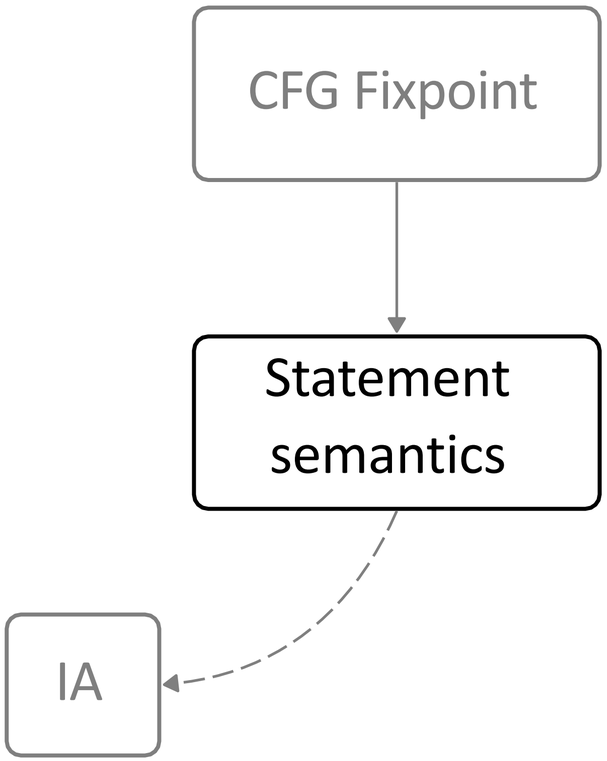
Syntax vs Semantics



We want to specify semantics **generically**

- It should apply to all analyses

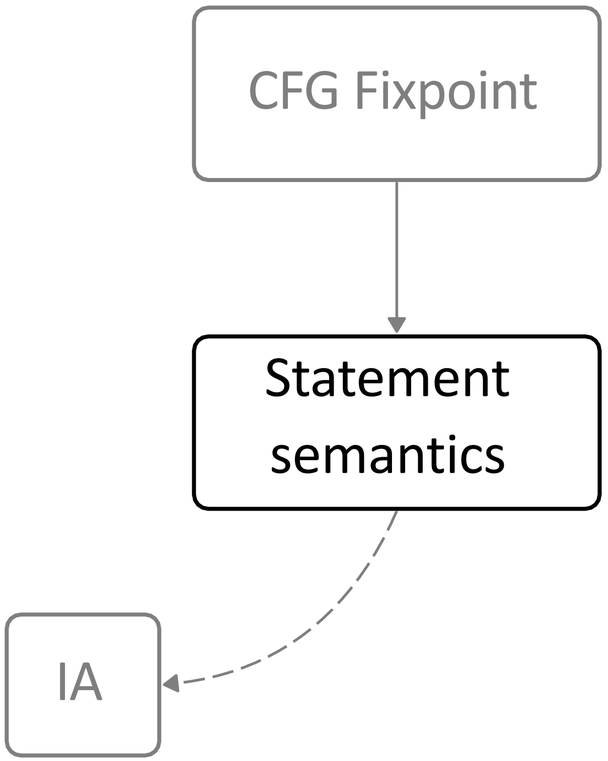
Syntax vs Semantics



We want to specify semantics **generically**

- It should apply to all analyses
- Analyses should not give meaning to the syntax

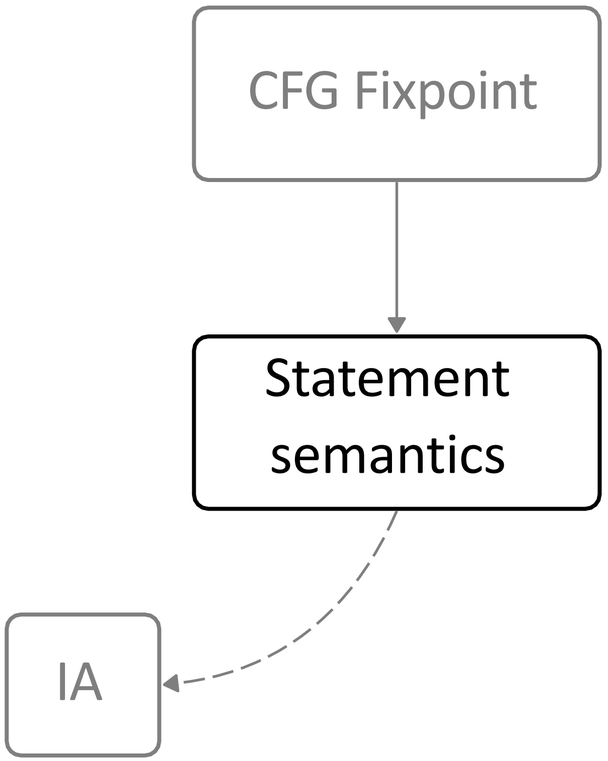
Syntax vs Semantics



We want to specify semantics **generically**

- It should apply to all analyses
- Analyses should not give meaning to the syntax
  - Java's `a + b` can be a sum or a concatenation

Syntax vs Semantics

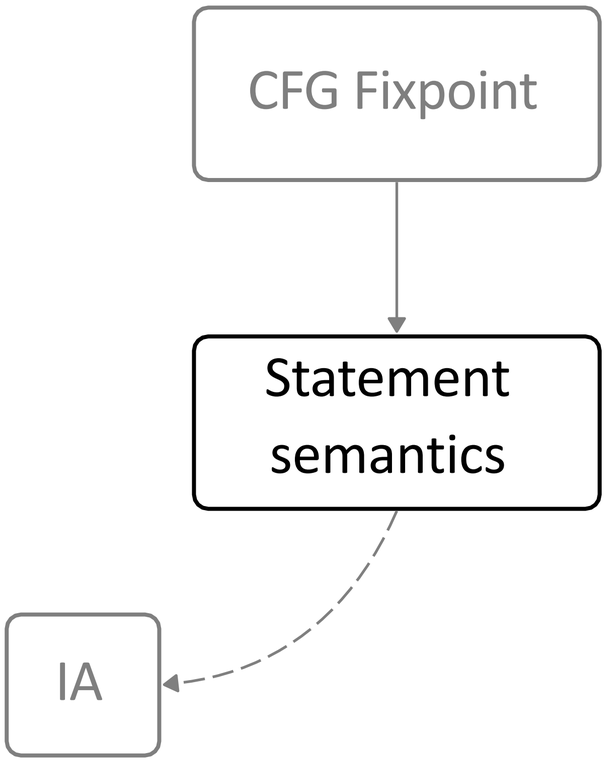


We want to specify semantics **generically**

- It should apply to all analyses
- Analyses should not give meaning to the syntax
  - Java's `a + b` can be a sum or a concatenation

One solution: rewrite to an IR

Syntax vs Semantics



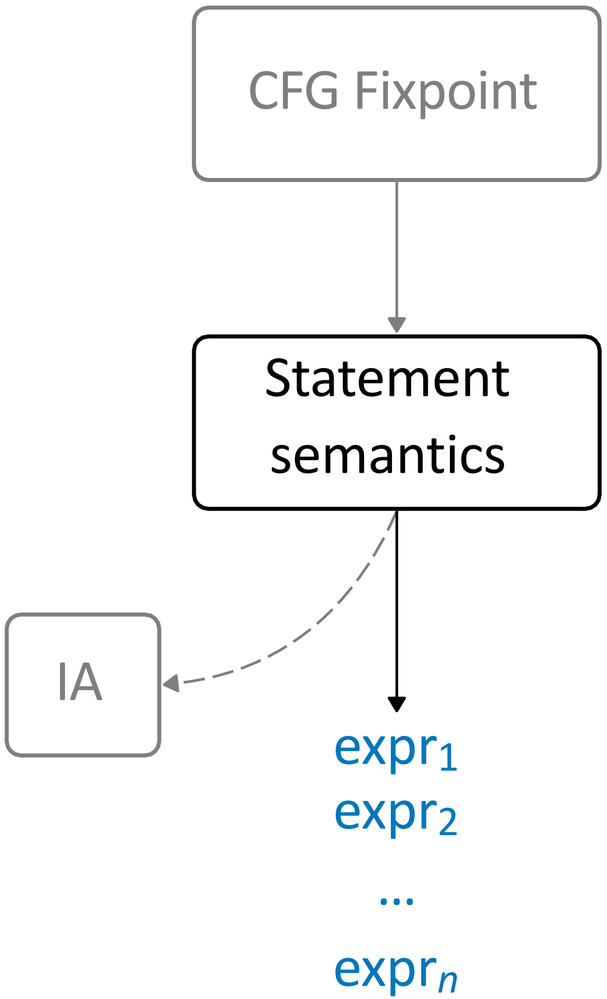
We want to specify semantics **generically**

- It should apply to all analyses
- Analyses should not give meaning to the syntax
  - Java's `a + b` can be a sum or a concatenation

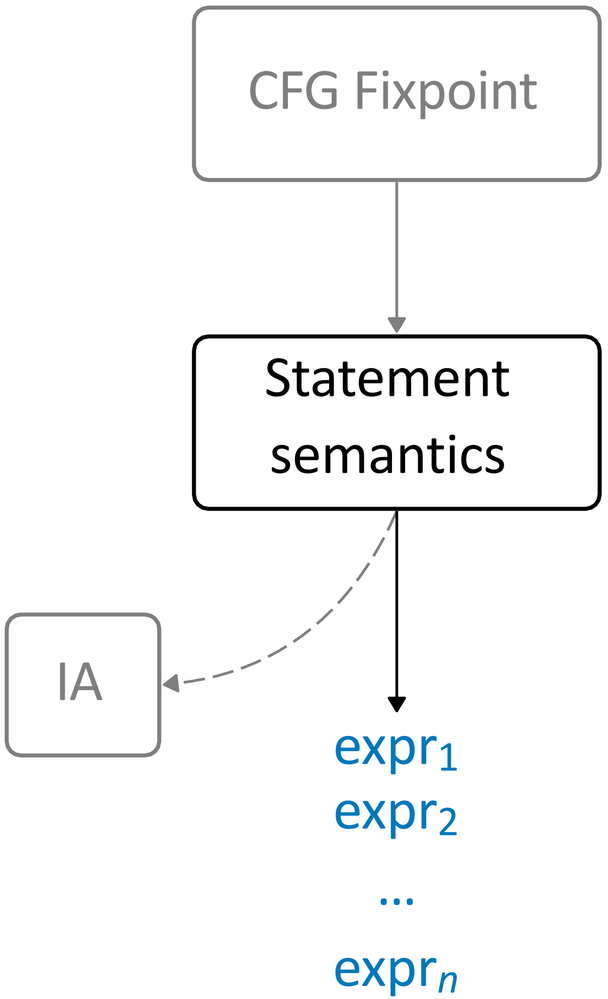
One solution: rewrite to an IR

But we might want to exploit semantic information

Symbolic Expressions



Symbolic Expressions

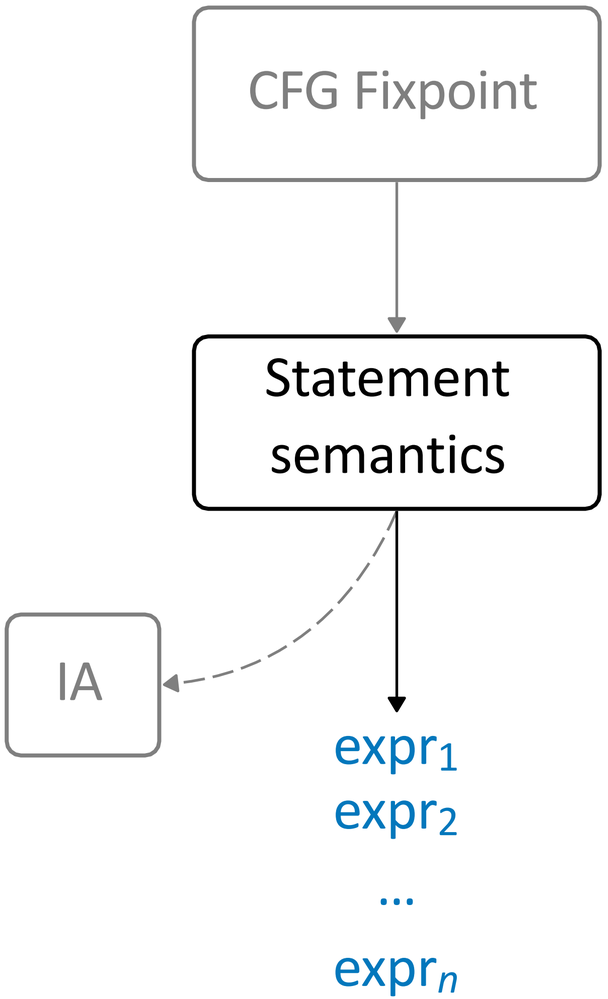


Our solution: break Statements into SymbolicExpressions

The rewriting happens dynamically, and can exploit the analysis

- Types, possible values, …

Symbolic Expressions



Our solution: break Statements into SymbolicExpressions

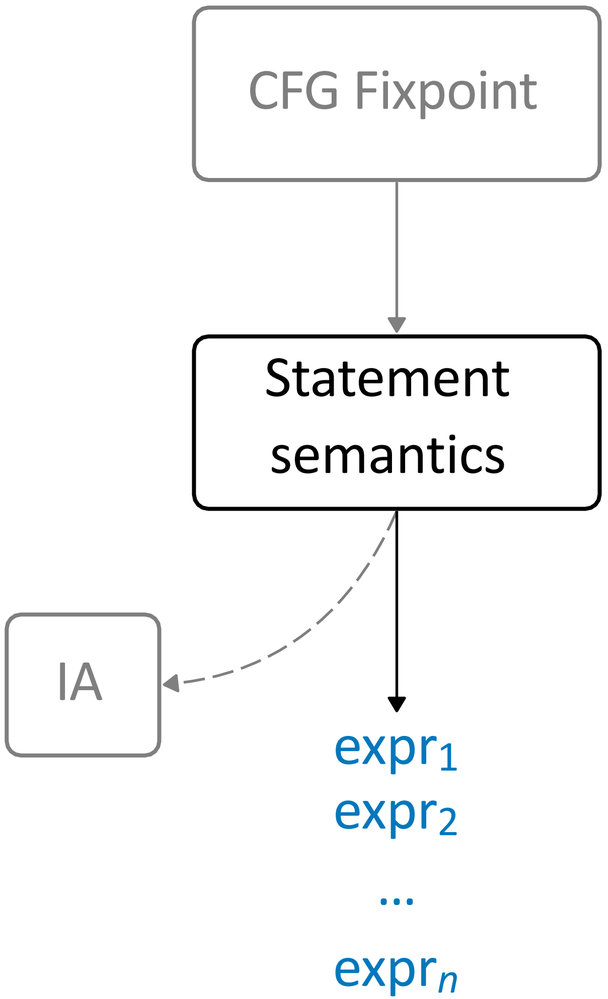
The rewriting happens dynamically, and can exploit the analysis

- Types, possible values, …

SymbolicExpressions are extensible

- We provide some, but users can define more

Symbolic Expressions



Our solution: break Statements into SymbolicExpressions

The rewriting happens dynamically, and can exploit the analysis

- Types, possible values, …

SymbolicExpressions are extensible

- We provide some, but users can define more

Analyses only have to model these expressions, ignoring the syntax behind them

Examples (Pseudocode)

```
Plus.semantics(): // left + right
 if type(left) is string or type(right) is string:
  return domain.eval(new BinaryExpression("cat", left, right))
 else:
  return domain.eval(new BinaryExpression("+", left, right))
```

Examples (Pseudocode)

```
Plus.semantics(): // left + right
 if type(left) is string or type(right) is string:
  return domain.eval(new BinaryExpression("cat", left, right))
 else:
  return domain.eval(new BinaryExpression("+", left, right))
```

```
Conditional.semantics(): // condition ? ifTrue : ifFalse
 sat = domain.isSatisfied(condition)
 tt = ifTrue.semantics()
 ff = ifFalse.semantics()
 return sat.isTrue() ? tt : (sat.isFalse() ? ff : tt ⊔ ff)
```

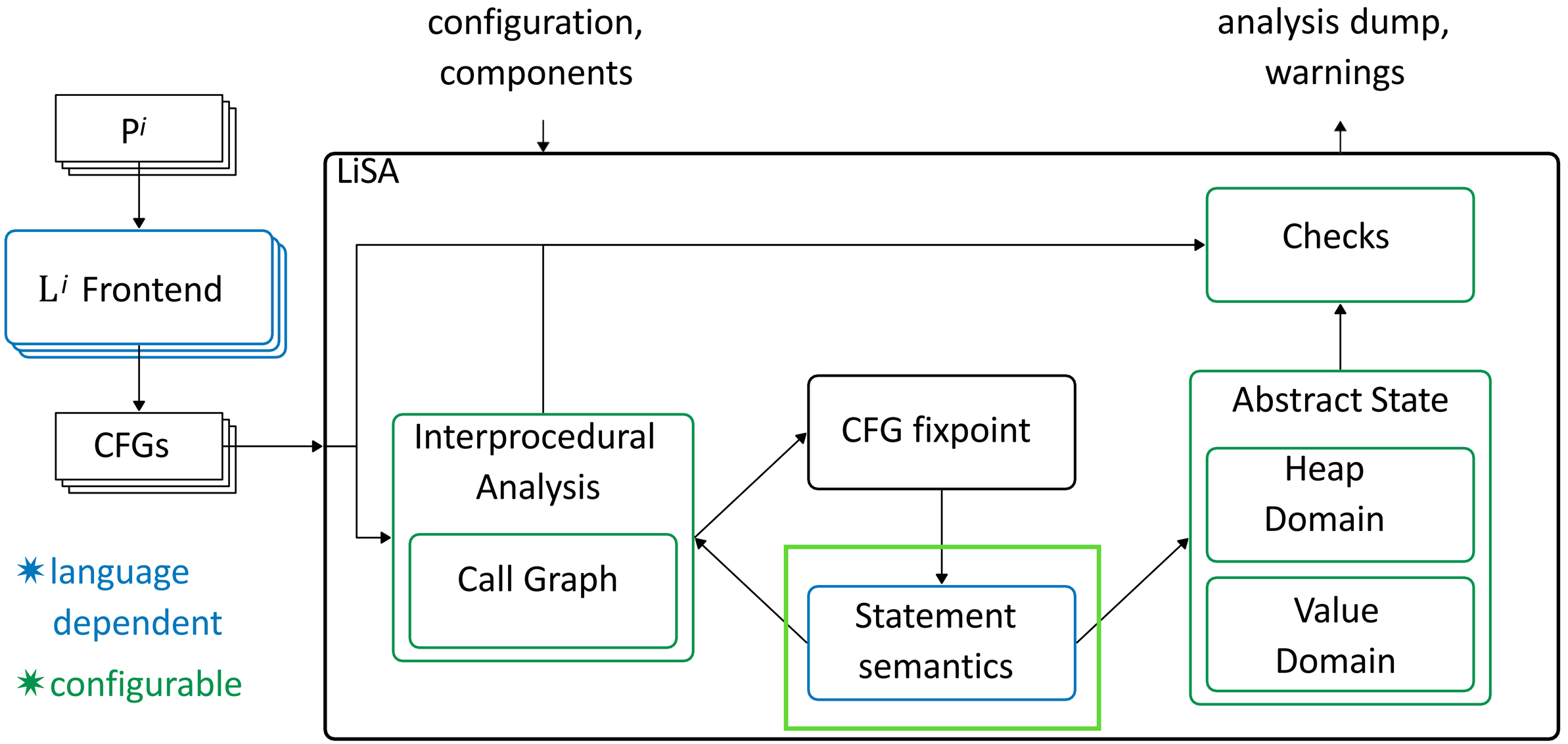
Examples (Pseudocode)

```
Plus.semantics(): // left + right
 if type(left) is string or type(right) is string:
  return domain.eval(new BinaryExpression("cat", left, right))
 else:
  return domain.eval(new BinaryExpression("+", left, right))
```

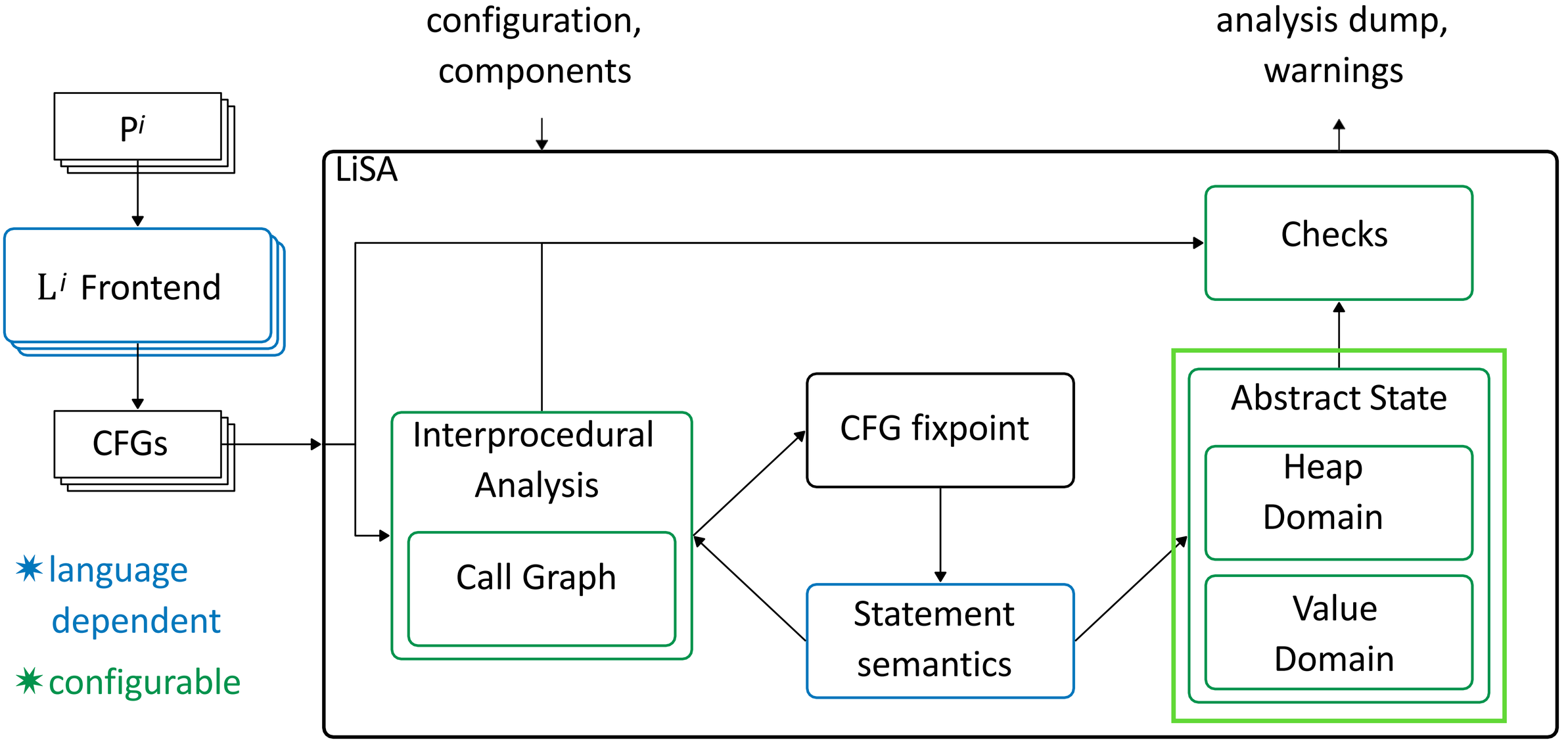
```
Conditional.semantics(): // condition ? ifTrue : ifFalse
 sat = domain.isSatisfied(condition)
 tt = ifTrue.semantics()
 ff = ifFalse.semantics()
 return sat.isTrue() ? tt : (sat.isFalse() ? ff : tt ⊔ ff)
```

```
PreIncrement.semantics(): // ++var
 add = new BinaryExpression("+", var, new Const(1))
 d1 = domain.store(var, add)
 return d1.eval(var)
```

LiSA Overview

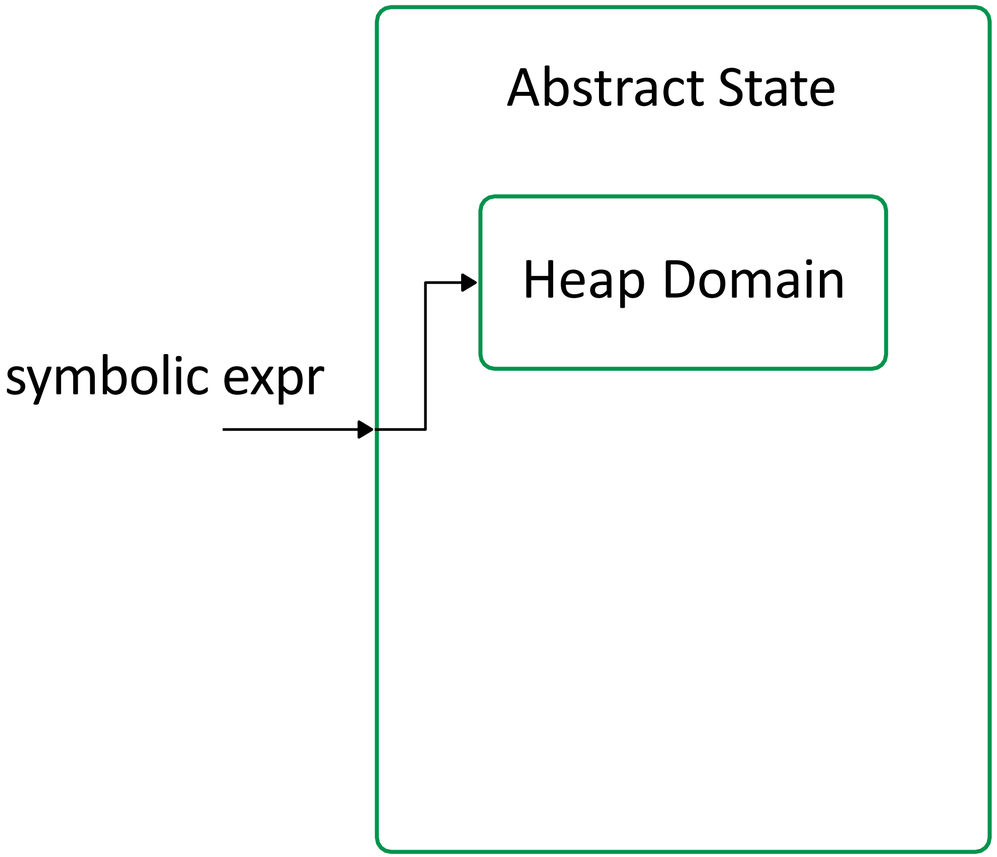


LiSA Overview



The Abstract State

[Fer16]



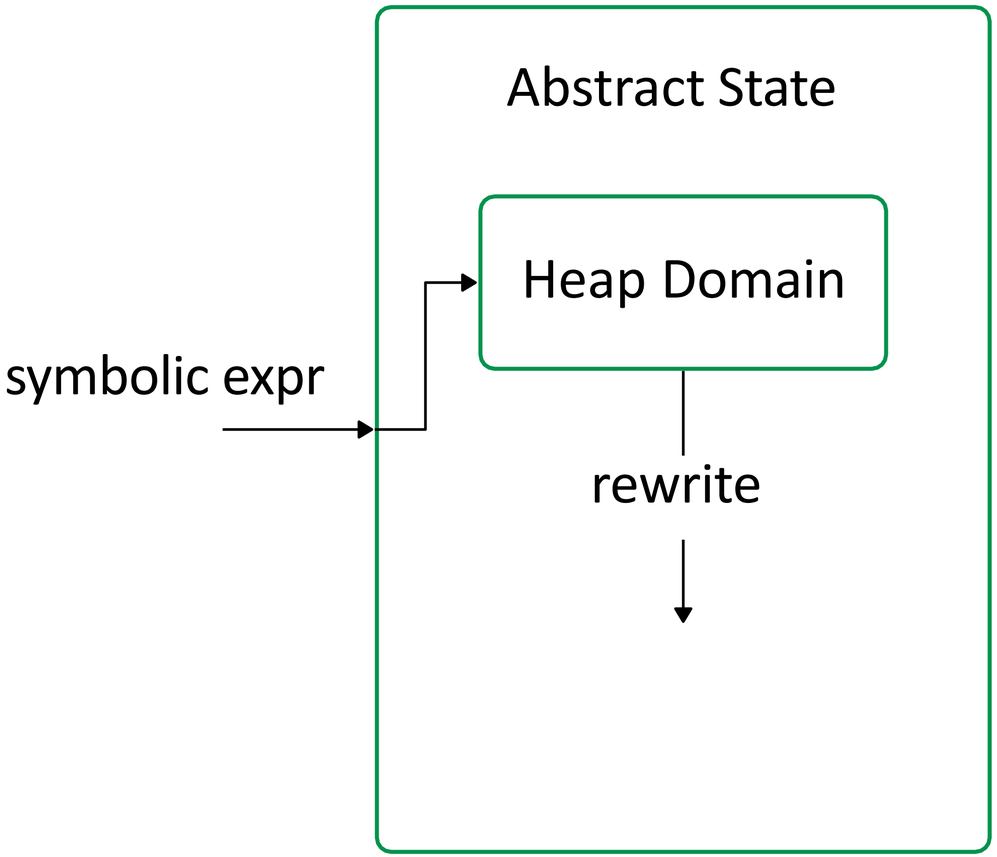
The Heap Domain tracks the **structure of the dynamic memory**

- new allocations
- pointer aliasing
- …

✷ configurable

The Abstract State

[Fer16]



✷configurable

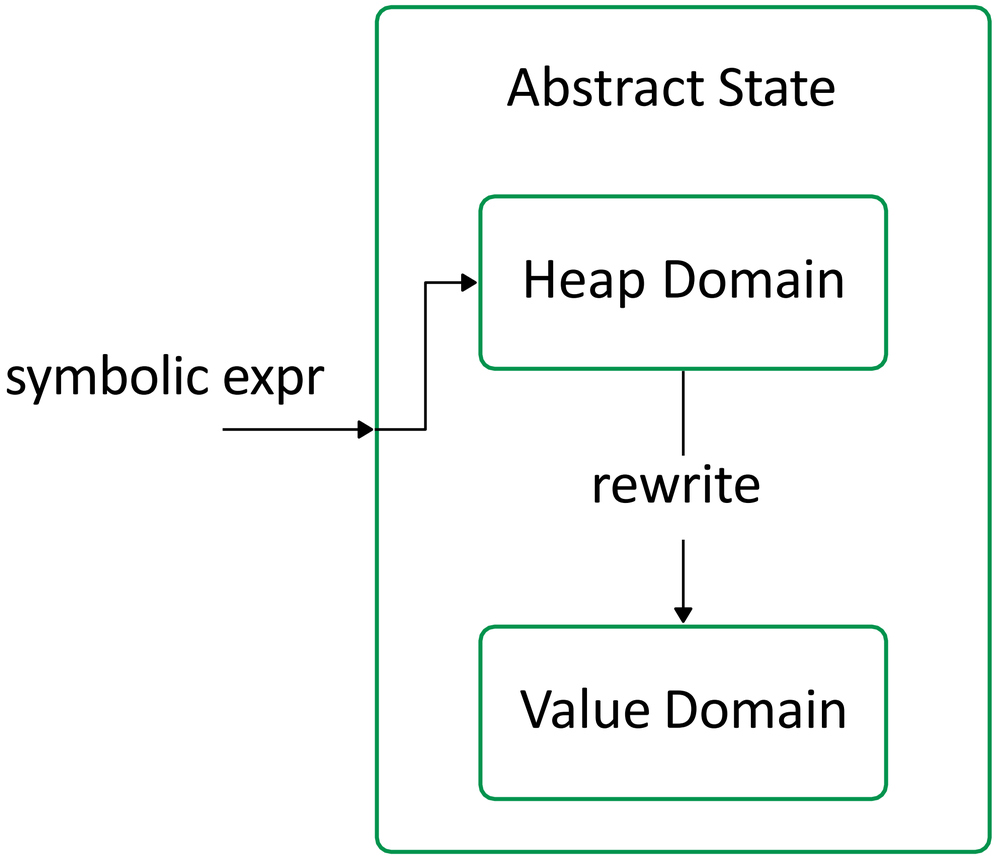
The Heap Domain tracks the **structure of the dynamic memory**

- new allocations
- pointer aliasing
- …

The Heap Domain **rewrites memory-related expressions** to their symbolic variables

The Abstract State

[Fer16]



✷configurable

The Heap Domain tracks the **structure of the dynamic memory**

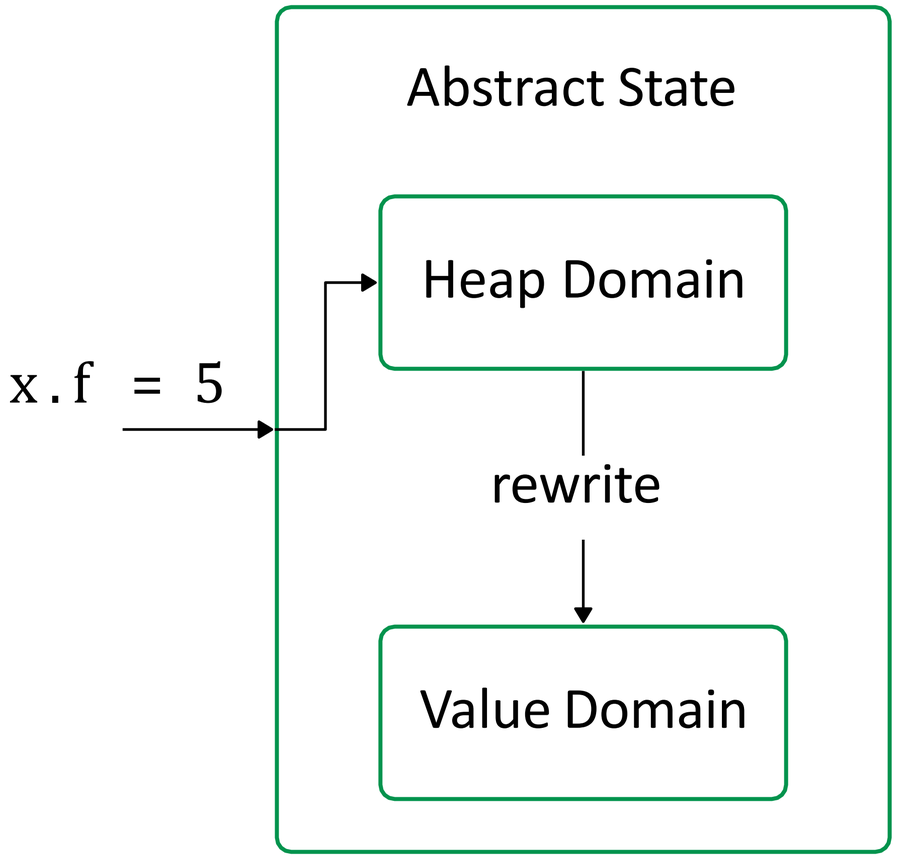
- new allocations
- pointer aliasing
- …

The Heap Domain **rewrites memory-related expressions** to their symbolic variables

The Value Domain only “sees” expressions with **variables and constants**

The Abstract State

[Fer16]



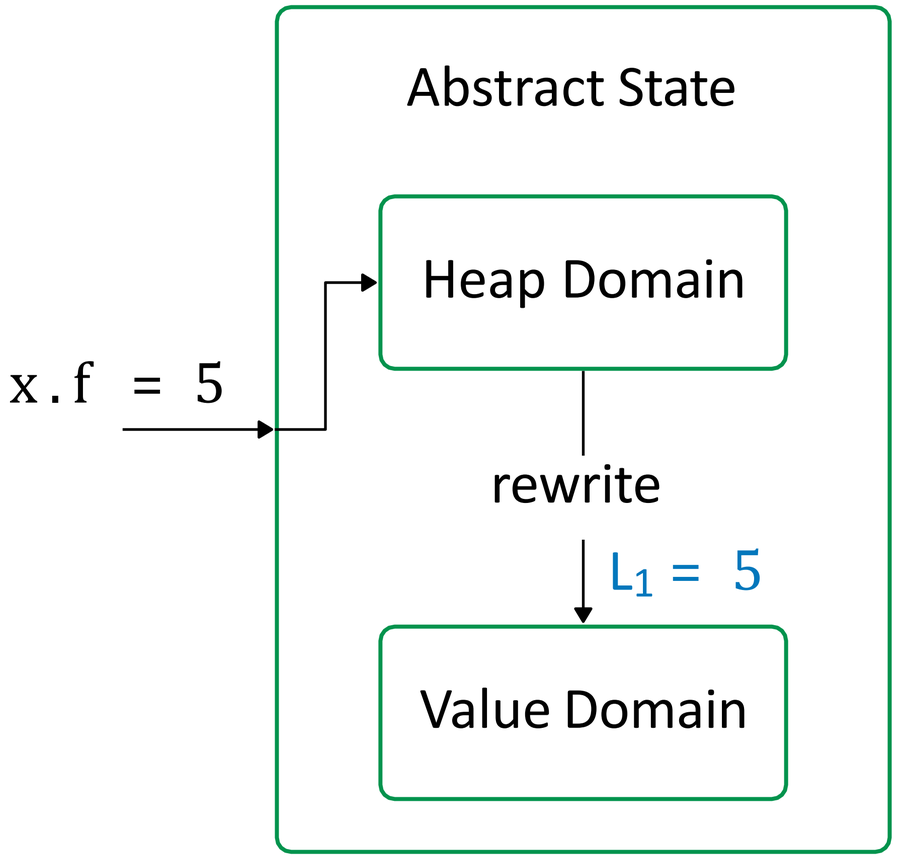
Heap Domain: $x \rightarrow L_1$

Value Domain:

✷configurable

The Abstract State

[Fer16]



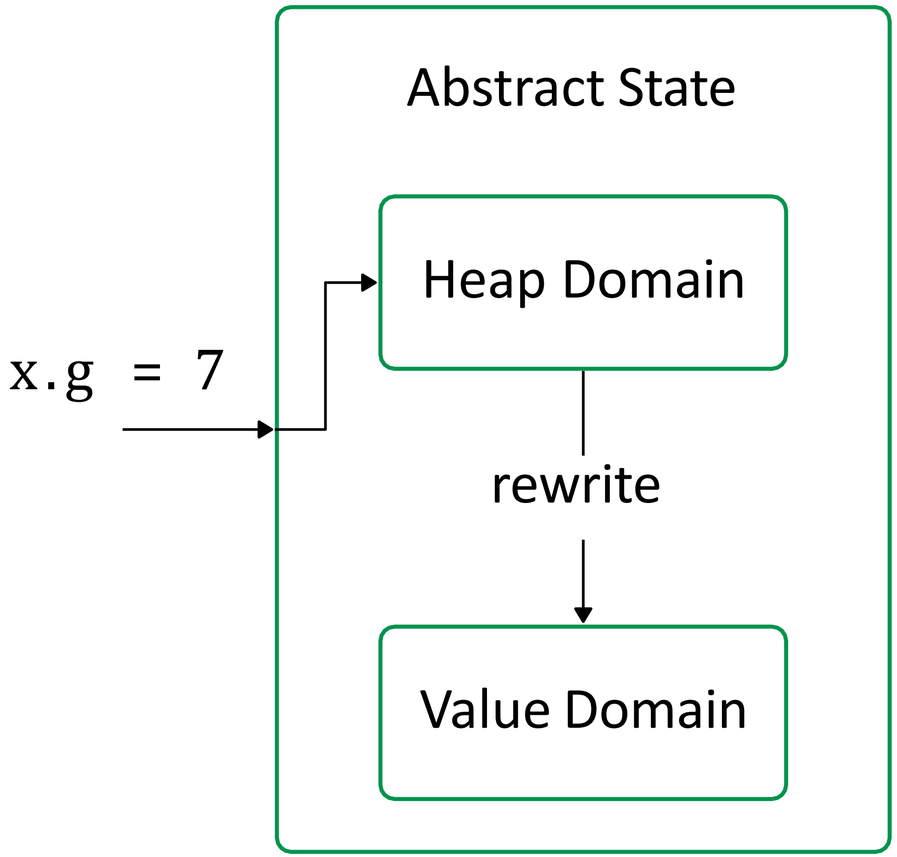
Heap Domain: $x \rightarrow L_1$

Value Domain: $L_1 \rightarrow \{5\}$

✷configurable

The Abstract State

[Fer16]



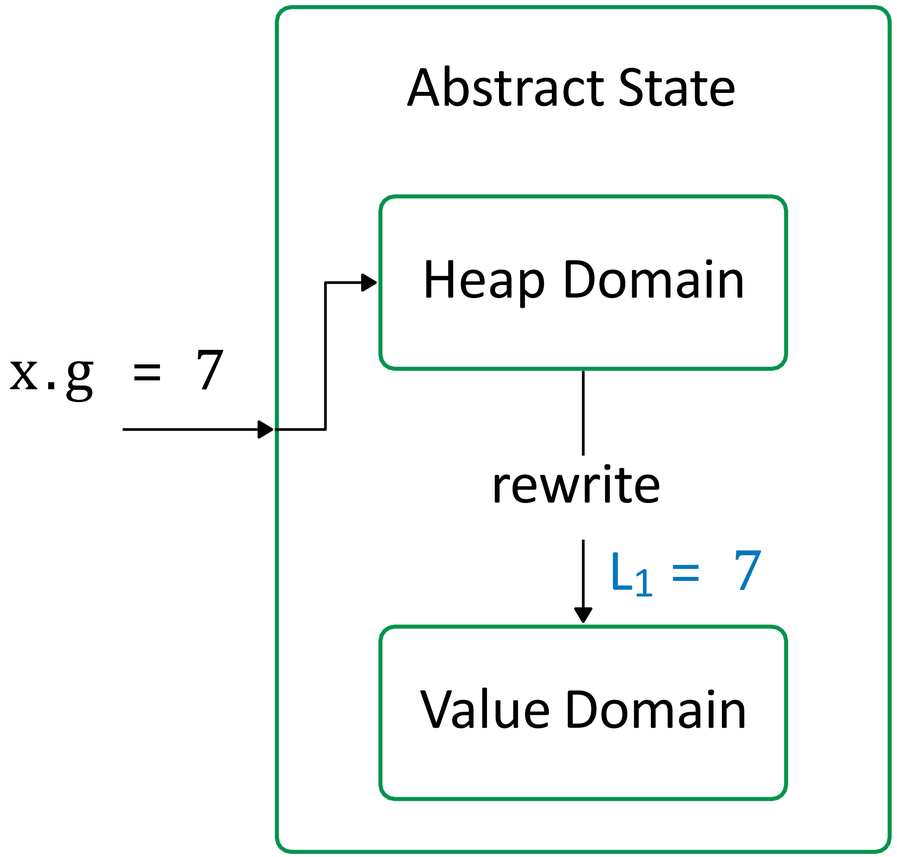
Heap Domain: $x \rightarrow L_1$

Value Domain: $L_1 \rightarrow \{5\}$

✷configurable

The Abstract State

[Fer16]



✷configurable

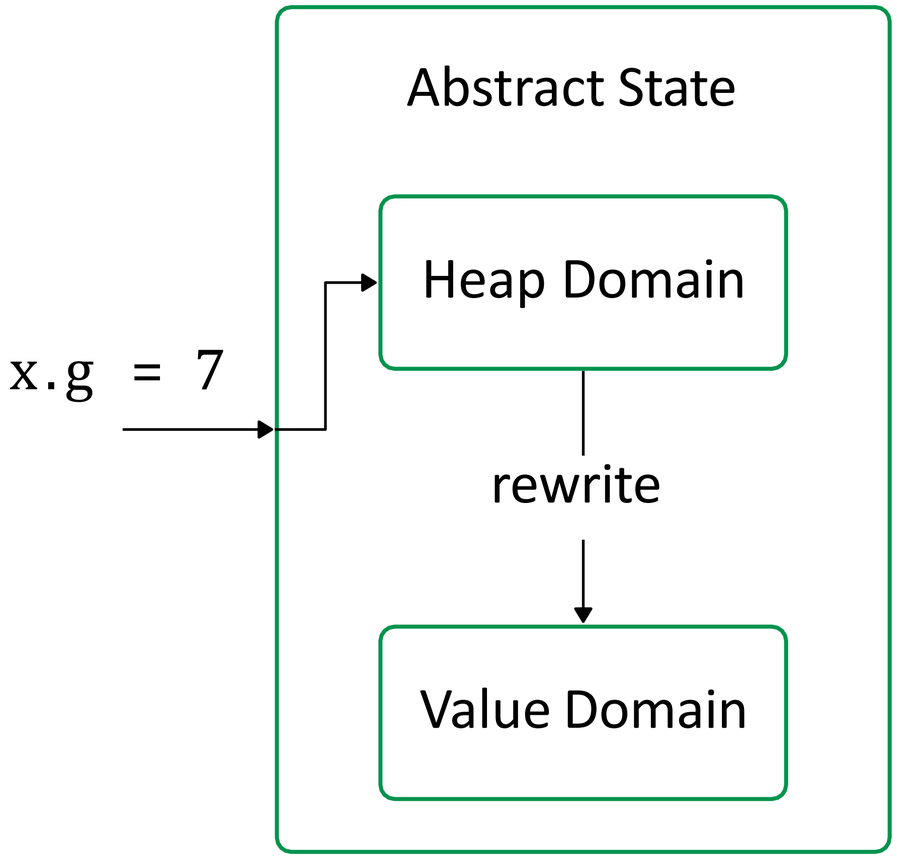
Heap Domain: $x \rightarrow L_1$

Value Domain: $L_1 \rightarrow \{5, 7\}$

Need to perform a weak update since x.f and x.g are abstracted to the same allocation site (field-insensitive)

The Abstract State

[Fer16]



Heap Domain: $x \rightarrow L_1$

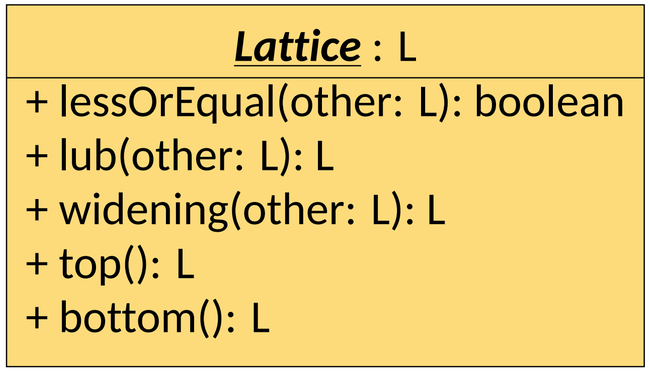
Value Domain: $L_{1.f} \rightarrow \{5\}, L_{1.g} \rightarrow \{7\}$

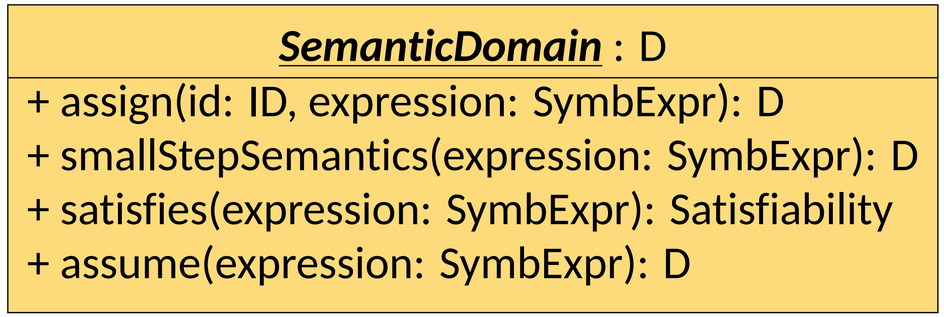
(field-sensitive call site based heap analysis)

✷configurable

Abstract domains in LiSA

Abstract states must be built, merged, and compared during the fixpoint





Abstract domains in LiSA

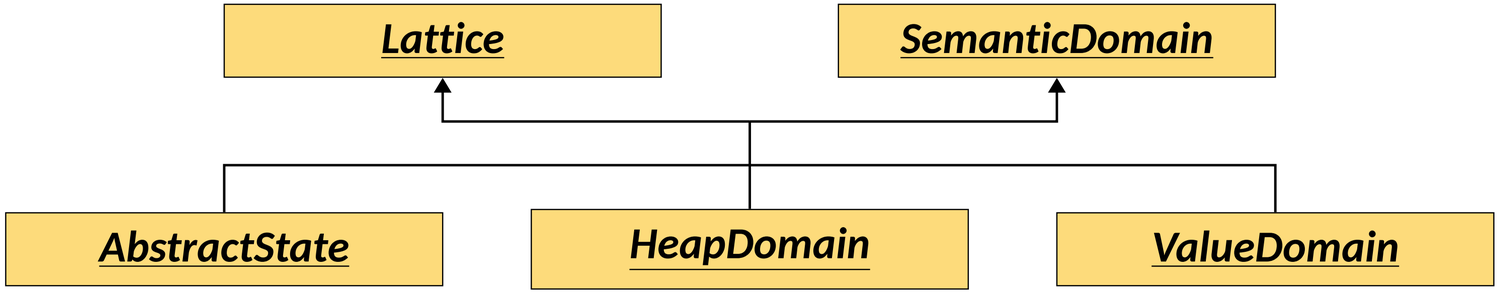
Abstract states must be built, merged, and compared during the fixpoint

Lattice : L
+ lessOrEqual(other: L): boolean + lub(other: L): L + widening(other: L): L + top(): L + bottom(): L

SemanticDomain : D
+ assign(id: ID, expression: SymbExpr): D + smallStepSemantics(expression: SymbExpr): D + satisfies(expression: SymbExpr): Satisfiability + assume(expression: SymbExpr): D

Abstract State, Value Domain, and Heap Domain all implement both!

Abstract domains in LiSA



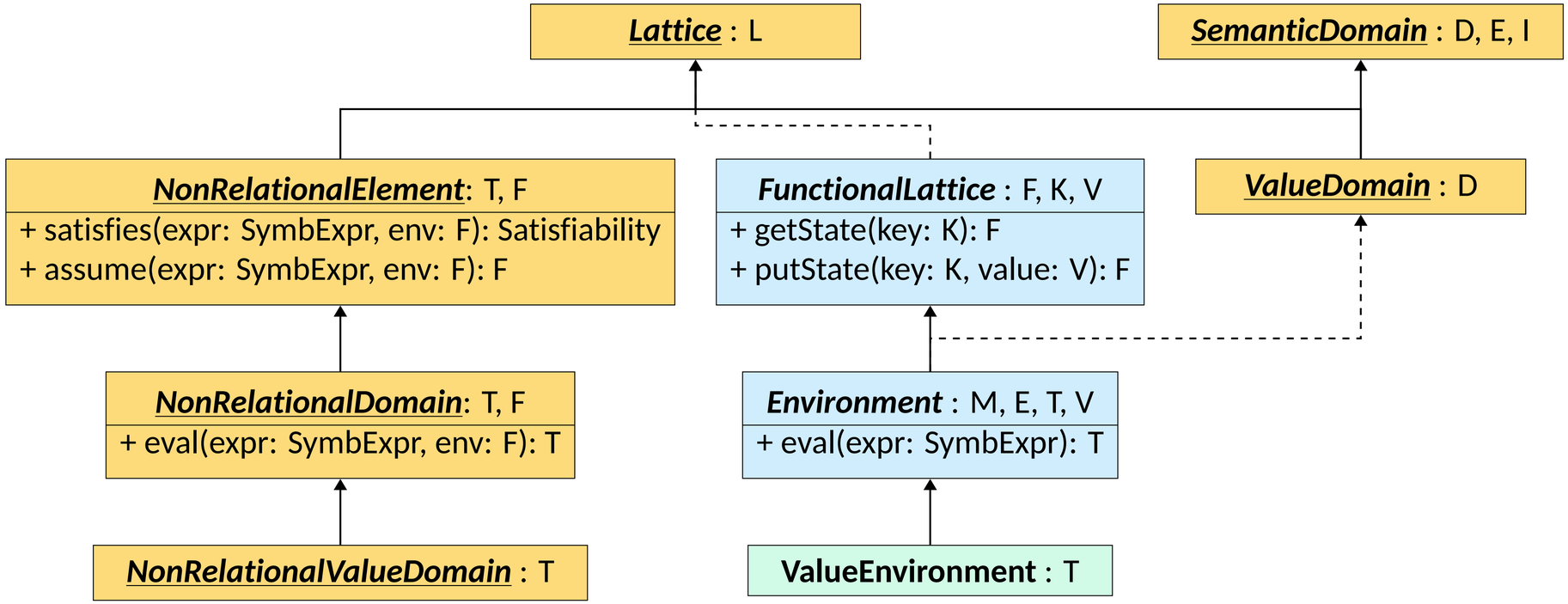
Each instance is a lattice element:

- Its state carries the abstract information
- Lattice operators compare/combine that information

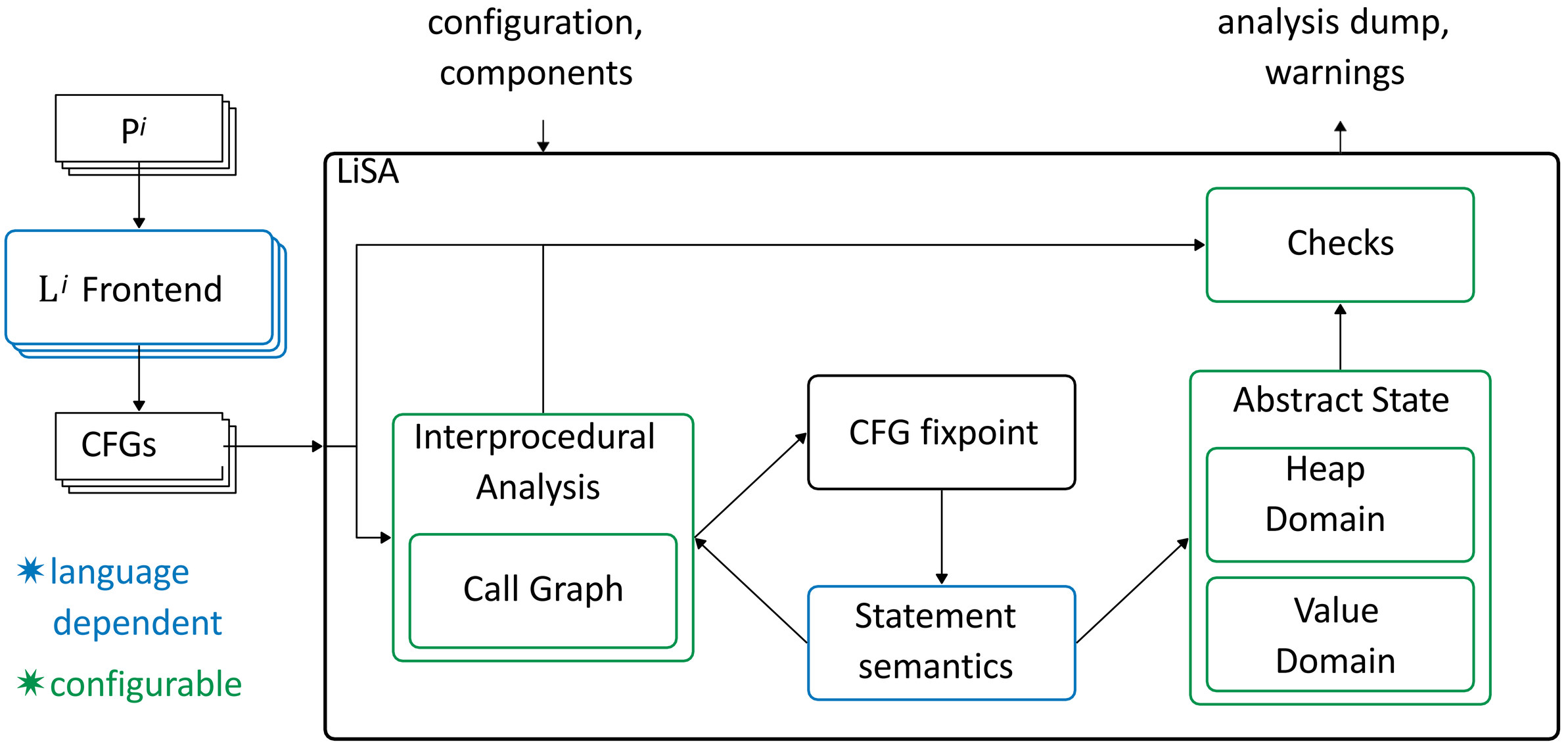
Each instance can be used as a pre-state:

- `SemanticDomain` operators can transform it into a post-state

Main classes for value analyses



LiSA Overview



LiSA Frontends

LiSA Frontends

Languages:

- Go
- Michelson
- EVM bytecode
- Python
- LLVM
- Java

LiSA Frontends

Languages:

- Go
- Michelson
- EVM bytecode
- Python
- LLVM
- Java

Topics:

- Blockchain
- Strings
- Dynamic languages
- Numeric trends
- Data science
- Modular interactions

LiSA Frontends

Languages:

- Go
- Michelson
- EVM bytecode
- Python
- LLVM
- Java

Topics:

- Blockchain
- Strings
- Dynamic languages
- Numeric trends
- Data science
- Modular interactions

Teaching:

- SCSR @ UNIVE: used for 4 years
- SASV @ UNIPR: used for 2 years

LiSA Frontends









LiSA Frontends

Languages:

- Go
- Michelson
- EVM bytecode
- Python
- LLVM
- Java









LiSA Frontends

Languages:

- Go
- Michelson
- EVM bytecode
- Python
- LLVM
- Java









Topics:

- Blockchain
- Strings
- Dynamic languages
- Numeric trends
- Data science
- Modular interactions

LiSA Frontends

Languages:

- Go
- Michelson
- EVM bytecode
- Python
- LLVM
- Java









Topics:

- Blockchain
- Strings
- Dynamic languages
- Numeric trends
- Data science
- Modular interactions

Teaching:

- SCSR @ UNIVE: used for 4 years
- SASV @ UNIPR: used for 2 years

GoLiSA

GoLiSA

- *Goal*: detecting non-determinism in Go smart contracts

GoLiSA

- *Goal*: detecting non-determinism in Go smart contracts

- *Why*: non-determinism in blockchain software do not allow nodes to reach a common consensual state ($\Longrightarrow$ forks, halt, services denied)

GoLiSA

- *Goal*: detecting non-determinism in Go smart contracts
- *Why*: non-determinism in blockchain software do not allow nodes to reach a common consensual state ($\Longrightarrow$ forks, halt, services denied)
- *How*: non-interference analysis

GoLiSA

- *Goal*: detecting non-determinism in Go smart contracts
- *Why*: non-determinism in blockchain software do not allow nodes to reach a common consensual state ($\Longrightarrow$ forks, halt, services denied)
- *How*: non-interference analysis
  - tracking if non-det values flows in statements that may modify the blockchain global state

GoLiSA

- *Goal*: detecting non-determinism in Go smart contracts
- *Why*: non-determinism in blockchain software do not allow nodes to reach a common consensual state ($\Longrightarrow$ forks, halt, services denied)
- *How*: non-interference analysis
  - tracking if non-det values flows in statements that may modify the blockchain global state

```
func foo(id string) {

    var obj MyObject
    obj.ID = id
    obj.Date = time.Now()

    value, _ := json.Marshal(obj)

    store.Set(id, value)
}
```

GoLiSA

- *Goal*: detecting non-determinism in Go smart contracts
- *Why*: non-determinism in blockchain software do not allow nodes to reach a common consensual state ($\Longrightarrow$ forks, halt, services denied)
- *How*: non-interference analysis
  - tracking if non-det values flows in statements that may modify the blockchain global state

```
func foo(id string) {

    var obj MyObject
    obj.ID = id
    obj.Date = time.Now()

    value, _ := json.Marshal(obj)

    store.Set(id, value)
}
```

GoLiSA

- *Goal*: detecting non-determinism in Go smart contracts
- *Why*: non-determinism in blockchain software do not allow nodes to reach a common consensual state ($\Longrightarrow$ forks, halt, services denied)
- *How*: non-interference analysis
  - tracking if non-det values flows in statements that may modify the blockchain global state

```
func foo(id string) {

    var obj MyObject
    obj.ID = id
    obj.Date = time.Now()

    value, _ := json.Marshal(obj)

    store.Set(id, value)
}
```

Warning!

Tools/Analysis	#Warnings	#TP	#FP	#FN
GoLiSA	195	124	71	0
ChainCode	203	68	135	53
Revive^CC	351	79	272	1

EVMLiSA

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts

- The execution flow may be altered via `JUMP` instruction…

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts
- The execution flow may be altered via `JUMP` instruction…
- …but the location to which `JUMP` instructions jump is not hardcoded but they are dynamically computed

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts

- The execution flow may be altered via `JUMP` instruction…

- …but the location to which `JUMP` instructions jump is not hardcoded but they are dynamically computed

```
PUSH1 0x0A PUSH1 0x0C ADD JUMP
```

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts
- The execution flow may be altered via `JUMP` instruction…
- …but the location to which `JUMP` instructions jump is not hardcoded but they are dynamically computed

```
PUSH1 0x0A PUSH1 0x0C ADD JUMP
```

- *Goal*: build sound CFGs for EVM bytecode

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts
- The execution flow may be altered via `JUMP` instruction…
- …but the location to which `JUMP` instructions jump is not hardcoded but they are dynamically computed

```
PUSH1 0x0A PUSH1 0x0C ADD JUMP
```

- *Goal*: build sound CFGs for EVM bytecode
- *How*: parametric ℓ-sized abstract h-sized stack set domain

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts
- The execution flow may be altered via `JUMP` instruction…
- …but the location to which `JUMP` instructions jump is not hardcoded but they are dynamically computed

```
PUSH1 0x0A PUSH1 0x0C ADD JUMP
```

- *Goal*: build sound CFGs for EVM bytecode
- *How*: parametric ℓ-sized abstract h-sized stack set domain
  - then, exploits the inferred stacks to add edges…

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts
- The execution flow may be altered via `JUMP` instruction…
- …but the location to which `JUMP` instructions jump is not hardcoded but they are dynamically computed

```
PUSH1 0x0A PUSH1 0x0C ADD JUMP
```

- *Goal*: build sound CFGs for EVM bytecode
- *How*: parametric ℓ-sized abstract h-sized stack set domain
  - then, exploits the inferred stacks to add edges…
  - … untile no new edges are discovered

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts
- The execution flow may be altered via `JUMP` instruction…
- …but the location to which `JUMP` instructions jump is not hardcoded but they are dynamically computed

```
PUSH1 0x0A PUSH1 0x0C ADD JUMP
```

- *Goal*: build sound CFGs for EVM bytecode
- *How*: parametric ℓ-sized abstract h-sized stack set domain
  - then, exploits the inferred stacks to add edges…
  - … untile no new edges are discovered
- *EVMLiSA checkers:* reentrancy, timestamp dependency, tx.origin

EVMLiSA

- EVM bytecode is a stack-based low-level running on the Ethereum Virtual Machine, the execution engine for Ethereum smart contracts
- The execution flow may be altered via `JUMP` instruction…
- …but the location to which `JUMP` instructions jump is not hardcoded but they are dynamically computed

```
PUSH1 0x0A PUSH1 0x0C ADD JUMP
```

- *Goal*: build sound CFGs for EVM bytecode
- *How*: parametric ℓ-sized abstract h-sized stack set domain
  - then, exploits the inferred stacks to add edges…
  - … untile no new edges are discovered
- *EVMLiSA checkers:* reentrancy, timestamp dependency, tx.origin

Tool	**Precision (%)**	**Recall (%)**	**F-measure (%)**
EVMLiSA	93.75	100.00	96.77
EtherSolve	85.71	60.00	70.59
Vandal	41.67	100.00	58.82
Mythril	21.01	96.67	34.52

LiSA latest news

LiSA latest news

Ricerca Premiata con Ca' Foscari

Analisi software: l'Università di Parma fra le prime nel mondo

Jlisa, un analizzatore del software sviluppato dall'Università di Parma e dall'Università Ca' Foscari di Venezia, ha ottenuto la medaglia di bronzo nella track Java di Sv-Comp, la maggiore competizione mondiale del settore. Un risultato eccezionale, frutto di un lavoro iniziato sei anni fa all'Università Ca' Foscari e ora svolto in collaborazione con l'Università di Parma. Nel team per Unipr Vincenzo Arceri, ricercatore di Informatica al Dipartimento di Scienze Matematiche, Fisiche e Informatiche, e Filippo Bianchi, studente del corso di laurea magistrale in Scienze Informatiche, e per Ca' Foscari il gruppo Software and System Verification (Ssv), guidato dai docenti Agostino Cortesi e Pietro Ferrara, in particolare con i ricercatori Luca Negrini e Luca Olivieri e i dottorandi Giacomo Zanatta e Teodors Lisovenko.

La Competition on Software Verification (Sv-Comp) nasce nel 2012 per iniziativa di Dirk Beyer, professore ordinario all'Università Ludwig Maximilian di Monaco di Baviera e tra i massimi esperti del settore. Negli anni ha raggiunto una sempre maggiore popolarità e attualmente è l'evento di riferimento del settore. I risultati di questa competizione vengono presentati a Tacas (una delle maggiori conferenze mondiali in ambito di verifica software, che quest'anno si terrà a Torino ad aprile), con la partecipazione di circa un centinaio di tecnologie diverse sviluppate a livello mondiale.

Jlisa ha partecipato alla track su programmi Java, ottenendo uno stupefacente terzo posto: un risultato davvero eccellente, considerato che si trattava della prima partecipazione di questo strumento. Si tratta inoltre dell'unica tecnologia 100% made in Italy che ha partecipato quest'anno a Sv-Comp.



Vincenzo Arceri
Ricercatore

Jlisa è strumento di analisi statica basato sul framework LiSa - Library for Static Analysis, sviluppato sempre da ricercatori di Unipr e Ca' Foscari. LiSa è oggi utilizzato dall'Università di Parma e dal team Ssv di Ca' Foscari sia in attività di ricerca, con una produzione scientifica consolidata, sia nella didattica, in particolare nei corsi dedicati all'analisi statica e alla verifica del software, contribuendo alla formazione di competenze avanzate in un settore cruciale per l'affidabilità dei sistemi digitali.

r.c.

© RIPRODUZIONE RISERVATA

PARMATODAY

Redazione

23 dicembre 2025 16:47

Si parla di

università

ATTUALITÀ

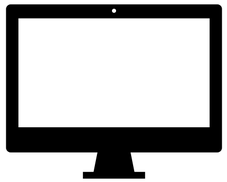
La National Security Agency Usa sceglie un sistema di verifica software targato Università di Parma e Ca' Foscari

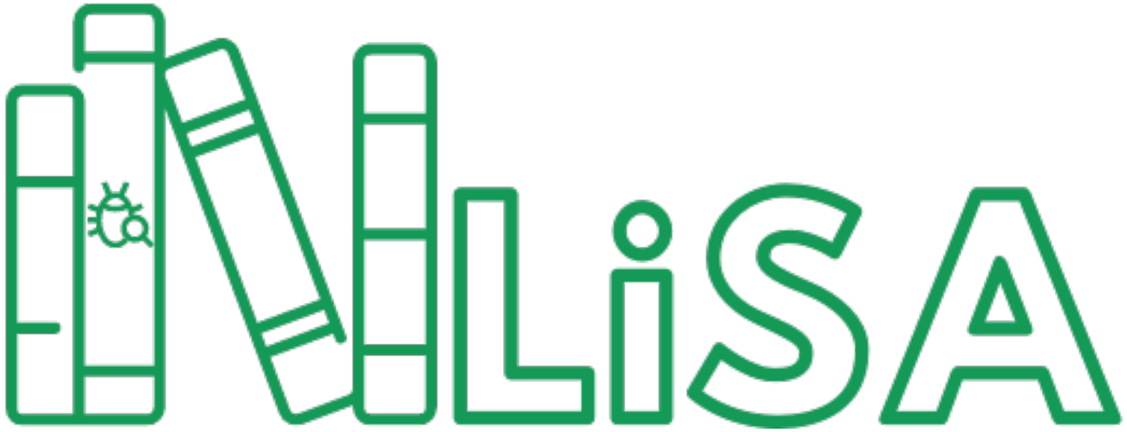
LiSA - Library for Static Analysis, progetto guidato dai ricercatori Luca Negrini (Ca' Foscari), Vincenzo Arceri (Unipr) e Luca Olivieri (Ca' Foscari)



Si chiama LiSA - Library for Static Analysis, ed è un sistema di verifica del software sviluppato da ricercatori dell'Università di Parma e dell'Università Ca' Foscari di Venezia. Un sistema che l'NSA (la National Security Agency americana) ha deciso di utilizzare per alcune attività. LiSA è un progetto di ricerca iniziato a Ca' Foscari dal 2020 e attualmente sviluppato e mantenuto da Ca' Foscari e Università di Parma, principalmente da Luca Negrini (Ca' Foscari), Vincenzo Arceri (Unipr) e Luca Olivieri (Ca' Foscari).

Example: sign analysis





https://github.com/lisa-analyzer

References

LiSA

- P. Ferrara, L. Negrini, V. Arceri, A. Cortesi: Static analysis for dummies: experiencing LiSA. SOAP@PLDI 2021: 1-6

- L. Negrini, P. Ferrara, V. Arceri, A. Cortesi: LiSA: A Generic Framework for Multilanguage Static Analysis. Challenges of Software Verification 2023: 19-42

- L. Negrini, V. Arceri, L. Olivieri, A. Cortesi, P. Ferrara: Teaching Through Practice: Advanced Static Analysis with LiSA. FMTea 2024: 43-57

EVMLiSA

- V. Arceri, M. Merenda, G. Dolcetti, L. Negrini, L. Olivieri, E. Zaffanella: Towards a Sound Construction of EVM Bytecode Control-Flow Graphs. FTfJP@ECOOP 2024: 11-16

- V. Arceri, M. Merenda, L. Negrini, L. Olivieri, E. Zaffanella: EVMLiSA: Sound Static Control-Flow Graph Construction for EVM Bytecode, Blockchain: Research and Applications, 2025

References

GoLiSA

- L. Olivieri, F. Tagliaferro, V. Arceri, M. Ruaro, L. Negrini, A. Cortesi, P. Ferrara, F. Spoto, E. Talin: Ensuring determinism in blockchain software with GoLiSA: an industrial experience report. SOAP@PLDI 2022: 23-29

- L. Olivieri, L. Negrini, V. Arceri, F. Tagliaferro, P. Ferrara, A. Cortesi, F. Spoto: Information Flow Analysis for Detecting Non-Determinism in Blockchain. ECOOP 2023: 23:1-23:25

- L. Olivieri, L. Negrini, V. Arceri, B. Chachar, P. Ferrara, A. Cortesi: Detection of Phantom Reads in Hyperledger Fabric. IEEE Access 12: 80687-80697 (2024)

- L. Olivieri, L. Negrini, V. Arceri, P. Ferrara, A. Cortesi, F. Spoto: Static Detection of Untrusted Cross-Contract Invocations in Go Smart Contracts. SAC 2025: 338-347

- L. Olivieri, L. Negrini, V. Arceri, P. Ferrara, A. Cortesi: Detection of Read-Write Issues in Hyperledger Fabric Smart Contracts. SAC 2025: 329-337

References

- V. Arceri, I. Mastroeni: Analyzing Dynamic Code: A Sound Abstract Interpreter for Evil Eval. ACM Trans. Priv. Secur. 24(2): 10:1-10:38, 2021

- V. Arceri, I. Mastroeni: A sound abstract interpreter for dynamic code. SAC 2020: 1979-1988

- V. Arceri, I. Mastroeni, S. Xu: Static Analysis for ECMAScript String Manipulation Programs. Applied Science, 2020

- V. Arceri, I. Mastroeni, Static Program Analysis for String Manipulation Languages. VPT@Programming, 2019



Zulip workspace on abstract interpretation

absint.zulipchat.com